

Received 17 October 2025, accepted 5 November 2025, date of publication 12 November 2025,
date of current version 21 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3631956

 SURVEY

A Survey on Learning-Based PnR Optimization for VLSI Physical Design Automation

R. JENILA¹ AND K. JAGANNADHA NAIDU², (Senior Member, IEEE)

Department of Micro and Nano Electronics, School of Electronics Engineering, Vellore Institute of Technology, Vellore 632014, India

Corresponding author: K. Jagannadha Naidu (jagannadhanaidu.k@vit.ac.in)

This work was supported by the Vellore Institute of Technology.

ABSTRACT The use of Artificial Intelligence (AI) is increasing nowadays in every industry because of its decision-making process, high computational speed, accurate predictions, and multi-classifications. The end goal of a machine learning algorithm is to draw reliable conclusions from unseen data by deploying trained models for a specific design. In Integrated Circuit (IC) physical design, Placement and Routing (PnR) are complicated processes that rely heavily on Performance, Power, and Area (PPA) metrics. Traditional heuristic-based approaches are often time consuming and may not scale efficiently with increasing design complexity. Therefore, the use of learning-based models in the layout process for PnR optimization can help reduce the production cost and design run time. This survey defines the PnR problem statement and provides an overview of traditional algorithms used in the PnR process. It addresses numerous aspects of learning-based techniques employed for PnR optimization based on various tasks. This survey also provides a review of the available benchmarks and inspects various challenges involved in learning-based PnR automation.

INDEX TERMS Artificial intelligence, deep learning, placement, physical design, routing, survey.

I. INTRODUCTION

The shrinkage in the size of Integrated Circuits (ICs) over the last few decades, along with the increase in components from several hundred to more than a billion on a chip, has driven the need for automation in design, layout, and verification processes [1]. Figure 1 represents the top-level IC design flow. In the first step, the design requirements and specifications are followed by a high-level representation of the IC using Hardware Descriptive Languages (HDLs) such as Verilog or VHDL. The logic synthesis converts the Register Transfer Level (RTL) representation into a gate-level netlist, which serves as the input for the physical design flow. In IC design, the Place and Route (PnR) process is a tedious and time consuming step in which commercial Electronic Design Automation (EDA) tools perform a lot of repetitive processes to meet the design requirements.

Physical design, often referred to as PnR or APR (Automated PnR), is the conversion from a synthesized

gate-level netlist to a layout. It is broadly categorized as a placement and routing problem [2]. During placement, the entire chip was partitioned into a number of modules with standard cells and macro blocks. Standard cells are predefined logic cells with fixed heights and varying widths depending on the functionality and drive strength of the module. Macros are larger than standard cells, often in custom shapes, and can be placed at any location in the core area. Each module has to be arranged as closely as possible within the core area to facilitate the routing phase [3].

Routing creates physical interconnections in the most efficient manner between the placed cells using metal layers by optimizing the design metrics. The challenge in routing is to satisfy complex design rules [4]. Global routing connects all pins of each net across different grid cells (G-cells), whereas detailed routing finalizes all necessary connections without violating any design rules. After routing, the resulting layout underwent physical verification to ensure that all specific design requirements were met. After physical verification, the layout is sent to the foundries in the GDSII file format for fabrication.

The associate editor coordinating the review of this manuscript and approving it for publication was Ludovico Minati³.

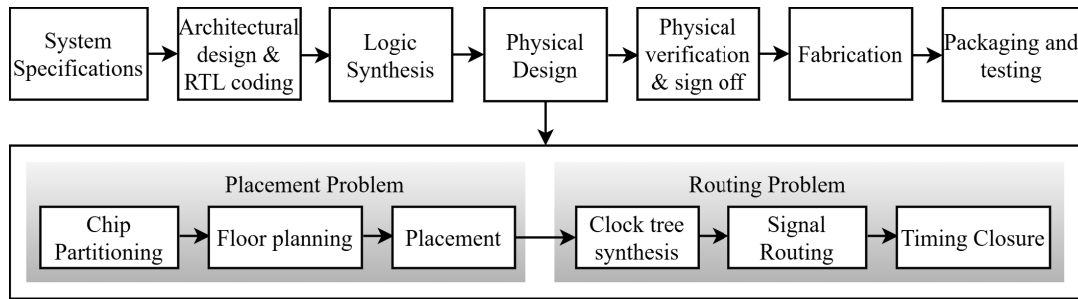


FIGURE 1. IC design flow from design specifications to timing closure.

The PnR stage is even more complicated because of the growing complexity of IC chips, which have billions of transistors and smaller technology nodes. Although traditional search and analytical-based algorithms are mathematically rigorous and provide consistent results, they are less effective in handling scalability challenges for modern large-scale designs. These methods depend on repetitive trial-and-error loops, require significant manual intervention, and are limited in balancing Performance, Power, and Area (PPA) objectives. Recent advancements in Machine Learning (ML) and Deep Learning (DL)-based models have outperformed traditional algorithms in several EDA tool tasks for chip design and optimization [5].

ML is a self-learning algorithm that learns patterns from the input data and makes predictions based on its training. DL is a subclass of machine learning that employs neural network architectures to train on large datasets [6]. There are different types of ML-DL algorithms based on their learning. Supervised learning trains ML algorithms using labeled datasets for both classification and regression tasks. In deep supervised learning, neural network architectures are employed to train a large set of labeled data. Unsupervised learning uses unlabeled datasets to train ML algorithms to identify similar patterns in the input data. Semi-supervised learning uses both labeled and unlabeled data to train the ML model. Deep unsupervised and semi-supervised learning employs neural network architectures to handle large datasets with no labels or only a limited amount of labeled data. In Reinforcement Learning (RL), an agent learns to make their own decisions by interacting with the environment and receiving feedback in the form of penalties for its actions. Deep Reinforcement Learning (DRL) integrates deep learning models with reinforcement learning and achieves high performance for complex tasks. Figure 2 depicts the most commonly used ML-DL algorithms.

The Convolutional Neural Network (CNN) is a fundamental DL-based model that uses convolution and pooling operations to efficiently extract spatial features and hierarchical representations from structured data to capture local patterns [7]. U-Net [8] extends CNNs with an encoder-decoder structure and skip connections that enable accurate spatial detail reconstruction, which is critical for

pixel-level prediction tasks. Graph Neural Networks (GNNs) operate on graph-structured data by sharing information among connected nodes and are effective for modeling relationships and interactions in complex networks [9]. Transformers use self-attention mechanisms to model global dependencies in input data, which scale well and capture long-range relationships [10]. Hybrid models [11] that unify diverse ML-DL algorithms to achieve better accuracy and robustness. These architectures can be employed for prediction and classification problems in EDA or as a standalone model to directly optimize design metrics like wirelength, congestion, delay, and power. This provides a promising solution by modeling complex design patterns, predicting potential issues at early stages, and automating tasks that require extensive engineering effort. As more domain-specific constraints are considered for the IC PnR process, it is also very challenging to train the Artificial Intelligence (AI) algorithms and find the best-fit model.

What does this survey adds?

- This survey provides a task-centric synthesis of learning-based approaches applied to the PnR process, presenting a more focused analysis compared to prior surveys.
- This survey organizes the existing work by key optimization objectives such as wirelength, routing congestion, design rule check violation, timing, IR drop, and power. This offers a clearer mapping between ML-DL algorithms and PnR challenges.
- Beyond listing related studies, this survey presents comparative tables with structured analysis in terms of datasets, input features, target labels, algorithms, training setups, and evaluation metrics.
- This survey examines the available benchmarks, analyzes scope and limitations, and highlights the open challenges such as data scarcity, scalability, and cross-node generalization.
- In addition, this survey outlines future research directions for developing more robust and adaptable learning-based PnR solutions.

The remainder of this paper is structured as follows: Section II defines the placement and routing problems with the traditional approaches used. Section III discusses various

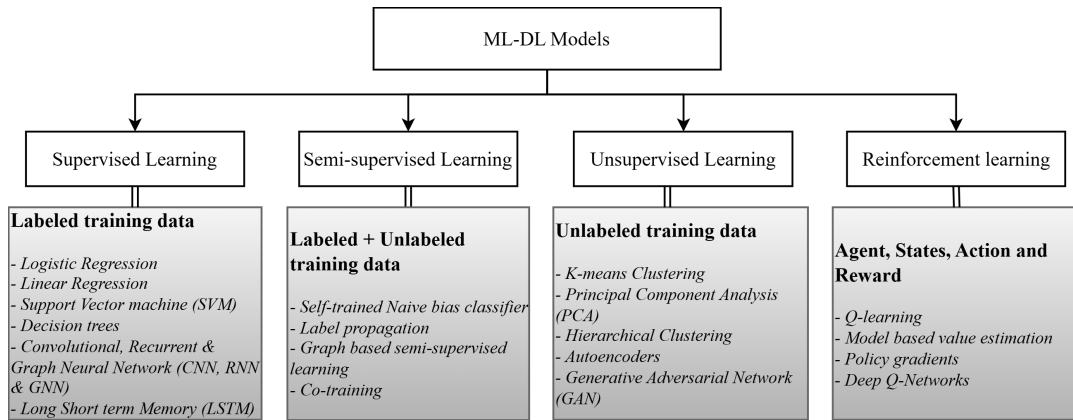


FIGURE 2. Commonly used ML-DL algorithms for various tasks based on different learning paradigms.

learning-based algorithms used for various tasks in PnR prediction and optimization. Section IV describes model training and performance evaluation metrics. Section V summarizes the available open-source benchmark suites. Section VI highlights the key challenges of using ML-DL models in PnR. Section VII outlines potential future research directions. The conclusion of this survey is given in Section VIII.

II. PROBLEM DEFINITION

This section provides an overview of placement and routing, including the definition and traditional approaches used for optimization.

A. PLACEMENT

The placement problem is formulated as a hypergraph G_p of (V_p, E_p) with a fixed die size and predefined pin locations, where V_p represents the blocks or modules $(V_{p1}, V_{p2}, \dots, V_{pn})$, and E_p represents the link between the blocks or modules. The modules should be placed efficiently by optimizing the design metrics such as wirelength, congestion, routability, power, and timing without any overlaps in the given core area.

One solution to this placement problem is the heuristic approach. Simulated Annealing (SA) [12], [13], Genetic Algorithm (GA) [14], Particle Swarm Optimization (PSO) [15], and Hopfield Neural Network [16] are heuristic approaches that have been employed for VLSI placement solutions. These algorithms are commonly used in placement problems because of their potential to handle small- to medium-sized problems and provide good-quality solutions within a reasonable time period. However, these search-based algorithms may produce suboptimal solutions in terms of total wirelength, area, latency, and power consumption. Some of them may face scalability issues and find it difficult to handle large-sized designs efficiently. In addition, VLSI placement involves some constraints, such as blockages and specified locations for certain components. This iterative-constructive method may struggle to tackle

placement constraints, leading to solutions with design rule violations.

Another approach is analytical, in which the placement problem is modeled as mathematical programming to optimize placement objectives and constraints [3]. SimPl, a self-contained, partition-based, and force-directed placement algorithm proposed by Kim et al. [17], can be implemented on multicore CPUs for thread-level and instruction-level parallelism. It is only suitable for standard cell placement, enabling quick implementation and efficient parallel processing. Figure 3 represents the SimPl algorithm, which employs a placement-dependent Bound-to-Bound (B2B) model that breaks down multi-pin nets into weighted edges based on the current placement and updates each iteration for the Half-Perimeter Wirelength (HPWL) approximation. The quadratic placement problem is efficiently solved using Conjugate Gradient (CG), which computes the lower-bound placement through quadratic wirelength minimization. Multilevel Adaptive Placement (MAPLE) [18], a placement algorithm, uses two-level clustering and Progressive Local Refinement (ProLR) in addition to SimPl iterations for mixed-size designs, thereby deriving the lower-bound placement and achieving a better trade-off between the wirelength and cell density. Polar [19], a quadratic global placer comprising Look-Ahead Rough Legalization (LAL), constructs an expansion region to enclose the hotspot where placement overflow occurs. It outperforms the existing state-of-the-art placers in terms of both runtime and placement quality.

RePlace [20], a Quadratic Programming (QP) model, achieves good routability-driven placement solutions and effective wirelength reduction through constraint-oriented local smoothing and dynamic step size adaptation. ePlace uses a placement density function by modeling the placement instance as an electrostatic system [21]. This algorithm addresses the high computational complexity of the non-linear approach and provides better placement solutions. The main drawbacks of these models are the low efficiency and longer convergence for large circuits. Several other analytic-based placers, such as Kraftwerk2

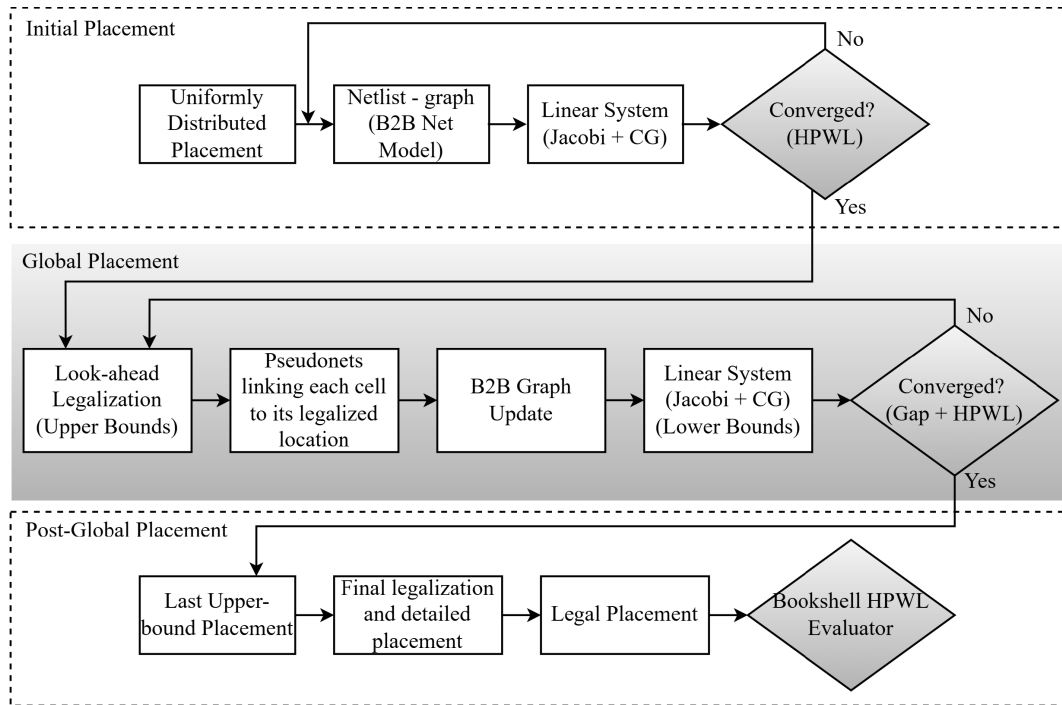


FIGURE 3. SimPI, a placement-dependent B2B model which is updated at each iteration after initial placement [17].

[22], FastPlace 3.0 [23], Relaxed Quadratic Spreading with Linearization (RQL) [24], APlace [25], Multilevel placement (mPL6) [26], and NTUPlace3 [27] have also been utilized for wirelength reduction and to avoid overlap between the cell modules. Analytic placers use a one-shot technique to place the macros and standard cells. The substantial growth in the use of macros owing to the widespread usage of IP modules incurs complexity and legal issues. This leads to the requirement of a desirable legalizer for macro placement. Thus, analytic placers are highly effective and provide high-quality solutions for medium-sized designs [28]. By making a parameter change in the constraints, it can be used to solve multi-objective optimization problems.

B. ROUTING

The routing problem is formulated as a grid graph G_r of (V_r, E_r) where V_r is the vertex that represents a specific rectangular cell (G-cell) of the chip $(V_{r1}, V_{r2}, \dots, V_{rn})$, and E_r represents the boundary of adjacent vertices (G-cells). There is a collection of Nets N_r , where each net represents a group of pins P_r . For each net, the process needs to find the path that connects all pins without overflow and to minimize the wirelength, vias, and DRVs. Figure 4 represents the simplified flow of the global routing process. The multi-pin nets are first decomposed into two-pin nets and routed using maze-based or pattern-based routing algorithms. The Rip-up and Reroute (RRR) process is applied iteratively to reduce congestion and improve routing quality.

Sequential and concurrent methods [29] are the traditional algorithms for solving routing problems. Maze Routing [30],

[31] is a simple grid-based search algorithm for Global Routing (GR) that determines the shortest path to interconnect the source and target pins. Later, Pattern Routing [32] was introduced, which follows predefined shapes such as the L-shape or Z-shape and becomes highly efficient. After initial routing, a Steiner Minimal Tree (SMT) method [33], [34], [35] was proposed to tackle highly congested areas. SMT adds intermediate Steiner points to reduce the total wirelength, which is ideal for multi-pin nets. Integer Linear Programming (ILP) [36], [37] models the routing problem mathematically and provides optimal solutions under given constraints using efficient ILP solvers.

Rip-up and Reroute (RRR) [38] is a heuristic approach that iteratively reroutes problematic nets to reduce design rule violations. For small-scale to mid-sized designs, these conventional methods are easy to implement, more flexible, minimize wirelength and provide optimal routing solutions. However, for large-scale designs, these approaches are computationally complex, do not handle congestion effectively, and introduce unnecessary detours during routing [39]. The Negotiation-based Rip-up and Reroute (N-RRR) method is a popular technique for the routing problem. FastRoute3.0 [40], MaizeRouter [41], NCTU-GR 2.0 [42], BoxRouter 2.0 [43], and FastRoute4.0 [44] are the various routers that use the N-RRR approach to address routing issues. These algorithms significantly improve scalability by constantly varying the routing cost function and make it more appropriate for large-scale designs.

The routing objectives are intrinsically linked to the interconnect delay and material properties. The delay, power, and crosstalk behavior in routed nets depend heavily on the

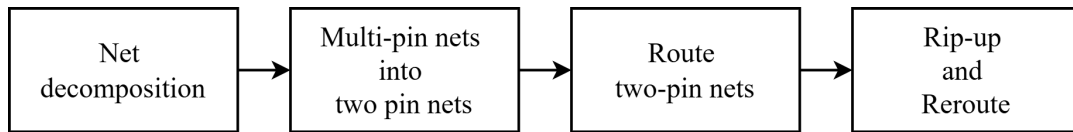


FIGURE 4. Basic step-by-step flow of global routing process.

Resistance, Inductance, and Capacitance (RLC) of interconnects, which are determined by wire geometry and material composition. Carbon NanoTube (CNT) bundles are used as alternatives to conventional *Cu* interconnects [45], because CNTs exhibit superior electrical, thermal, and reliability characteristics. In recent studies [46], a Tetramorphic Multi-Walled CNT (MWCNT) configuration was introduced with an RLC-based mathematical framework to analyze signal delay, crosstalk, and power across multiple technology nodes. Subsequently, Amin et al. proposed an adaptive mixed CNT bundle [47], combining multi-walled (MWCNTs) and single-walled CNTs (SWCNTs) to further minimize delay and crosstalk effects in high-speed interconnects. This provides a strong foundation for modeling technology-aware routing cost functions, as CNT geometry and material composition critically influence interconnect parasitics.

III. LEARNING-DRIVEN APPROACHES FOR PNR OPTIMIZATION

For decades, traditional PnR algorithms such as min-cut partitioning, search-based models, force-directed placement, maze or pattern routing, the Steiner tree mechanism, and RRR-route have played a huge role in the physical design process. These methods are built with strong mathematical foundations and are known to produce consistent solutions across different technology nodes. Unlike ML-DL algorithms, traditional methods offer greater interpretability, allowing designers to analyze cost functions and optimization procedures. However, these methods face certain limitations when applied to modern large-scale designs, which are summarized below.

- The increasing complexity of modern chips makes the classical approach impractical because of its excessive runtime and memory overhead.
- These algorithms focus on optimizing a single objective, such as wirelength or routability. However, achieving a balanced PPA remains a significant challenge.
- Traditional PnR methods are computationally inefficient and often require repetitive iterations that increase the design cost and turnaround time.
- The need for frequent manual intervention to adjust constraints, fine-tune parameters, and rerun processes delays design flow convergence.

In recent years, AI has been used most widely across industries, driven by its strength in making intelligent decisions, computational efficiency, accurate predictions, and ability to manage complex classification problems. Thus,

learning-based methods have been introduced to address the limitations of the traditional PnR approaches.

- ML-DL algorithms are capable of extracting complex patterns from design data. By analyzing prior outcomes, these models can identify trends and dependencies to improve the optimization quality.
- The ability of learning-based models to predict critical issues earlier minimizes the need for repetitive iterations at later stages of the design process.
- ML-DL models can be generalized across different designs and technology nodes, enabling the models to be more adaptable than classical algorithms.

Learning-based methods have been applied to various stages of the PnR flow, each addressing specific design challenges. This section organizes the existing literature based on various tasks, including wirelength, routing congestion, Design Rule Check (DRC), IR drop/power and timing prediction, and optimization. The reported results are obtained from different datasets and experimental setups used in previous studies. These values are included to highlight common methodologies and evaluation metrics rather than direct performance comparison.

A. WIRELENGTH OPTIMIZATION

Wirelength is the fundamental objective of PnR, and it directly influences delay, power, congestion, and overall design quality. During the PnR process, the wirelength is commonly estimated using the Half Perimeter Wirelength (HPWL) metric to approximate total interconnect length. It works by summing the half-perimeter of the bounding boxes that surround each net, providing a fast and effective estimation of the placement quality. Learning-based models explore supervised regression, neural networks, and reinforcement learning models to predict and optimize wirelength. The use of regression-based ML algorithms in [48] leverages the CAD Contest [49] and ISPD2006 [50] benchmarks to build predictive models for HPWL estimation in macro placement. The use of five different regression algorithms provides promising results in terms of wirelength prediction, improves overall efficiency, and reduces evaluation time. While boosted decision tree regression yields the lowest MAE, its performance is still constrained by its dependence on handcrafted features.

Graph-based learning models offer a richer representation of placement structure by encoding netlists as graphs or hypergraphs. GraphPlanner, a DL-based variational Graph Convolutional Network (GCN) model, uses a C-based clustering algorithm to optimize placement runtime and

wirelength reduction by partitioning the large-scale netlist into clusters based on the similarity of modules with minimized inter-cluster weighted connectivity [51]. The experimental analysis on various benchmarks and industrial designs results in an average 4 % reduction in wirelength and improves 25 % of placement runtime when compared with an existing mixed-size placer (RePlace). Also, GraphPlanner with DreamPlace achieves a 14% reduction in HPWL and a 2% placement runtime. Xu et al. [52] uses Graph Convolutional Network (GCN) coupled with RL for macro placement. The GCN encodes netlist graphs with block features such as area and aspect ratio. The RL policy gradient performs action for optimal placement with the negative rewards of HPWL, area, and overlap penalty. The experimental results on MCNC [53] and GSRC [54] show a consistent wirelength reduction of 3 – 12% compared to baseline models.

A Placement-aware Graph Neural Network (PL-GNN), a graph-based unsupervised learning method, builds cell clusters based on cell similarity [55]. The use of GNN and GraphSAGE converts the netlist to a hypergraph to initially develop node embeddings, and then k-means clustering for cell grouping leads to a suboptimal solution. The clustering results were then employed for optimization using a commercial placer. Figure 5 shows an unsupervised flow for placement optimization, where the PL-GNN performs grouping of cells based on structural and connectivity features. The experimental results on two multicore CPU designs with 28nm technology indicate that PL-GNN outperforms the existing modularity-based clustering and reduces 3.9 % wirelength, 2.8 % power, and 85.7% performance improvement compared to the traditional Synopsys ICC2 compiler placement flow. The PL-GNN can be used for any design, as it has the capability to learn the node representation for each design instance through the optimization of an unsupervised loss function. Recently, Lu et al. [56], introduced a deep graph clustering technique aimed at the proper placement of cells, incorporating design metrics directly as loss functions. The experimental results on five GPU/CPU designs at a 5nm technology node demonstrate that after the post-routing stage, this work achieves a reduction of 89 %, 26 %, 2.4 %, 1.4 % Total Negative Slack (TNS), frequency, wirelength, and clock power. Graph-based approaches scale well but generally involve greater computation and memory overhead compared to simpler regression techniques.

RL-based methods have emerged as a promising approach for directly optimizing multi-objective cost functions. Agnesina et al. [57] proposed Placement Parameter Optimizer (PPO)-based Deep Reinforcement Learning (DRL) for chip placement optimization, where RL can be employed for tuning the placement parameters. Figure 6 depicts the DRL flow in which the placement engine acts as the environment and the agent policy network generates actions to optimize the placement parameters. Using GNN embeddings, the implementation results under the TSMC 28nm process node demonstrate wirelength improvements of

up to 11% and 2.5%, on unseen netlists compared to domain expertise and a cutting-edge tool auto-tuner, respectively. This technique highlights the possibility of using automatic parameter adjustments to accelerate IC physical design flow. Amini et al. further extended RL with a Corner Block List (CBL) representation and Hypergraph Convolutional Networks (HyperGCN), enabling effective handling of large discrete action spaces in macro placement [58]. Although this requires longer training times, experimental results on MCNC and GSRC benchmark designs reported promising results over traditional approaches, with HPWL reduction up to 12% compared to the SA approach on large circuits. Table 1 provides an overview of the learning-based methods for wirelength optimization. This highlights the diversity of modeling strategies applied across various benchmarks along with input features and target objectives.

Figure 7 depicts the wirelength reduction percentage for the various learning-based models. The reported wirelength reduction ranges from (3 – 4%) to (10 – 12%), depending on the specific dataset and technology nodes. Although GNN and RL models demonstrate strong generalization to unseen designs, their computational overhead and integration into commercial placement flows remain an open challenge.

B. ROUTING CONGESTION PREDICTION

Routing Congestion (RC) has become an increasingly critical task in the PnR process, because congested regions make detailed routing more difficult. This occurs when the routing resource demand exceeds the available capacity, which results in increased wirelength, timing violations, and degraded design performance. Figure 8 depicts the traditional and ML-DL flows of RC prediction. The traditional method obtains congestion feedback after global routing and requires iterative placement-routing loops if the congestion exceeds acceptable limits. In ML-DL flow, congestion is predicted directly from placement features using trained models, enabling early hotspot detection, fewer design iterations and faster convergence to signoff. Learning-based RC prediction models aim to provide efficient and early identification of congestion hotspots in the IC design flow. Chen et al. [48] applied regression-based ML algorithms trained on a CAD contest and ISPD'06 benchmarks, using 16 spatial and connectivity features. Among the five models, the decision tree regression model performed best for RC estimation. Although the regression method is easy to interpret, it relies on handcrafted features and fails to capture complex patterns.

The convolutional layers in the CNN architectures are well-suited for identifying local spatial correlations, enabling the model to detect fine-grained congestion patterns. The ibU-Net model [60], a U-Net variant lightweight model, gives a significant improvement in RC and DRV prediction of 5% and 20% with less computational complexity compared to the existing method, RouteNet. An attention-based U-Net architecture with a standard deviation pooling mechanism was proposed in [61], which avoids overfitting during training and shows better performance in handling data

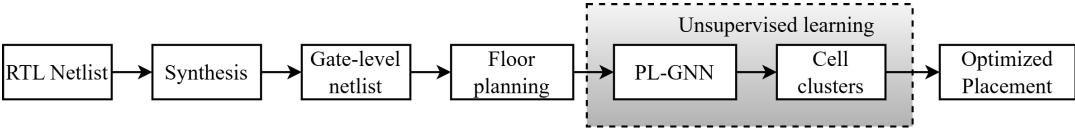


FIGURE 5. PL-GNN, an unsupervised placement optimization flow that reduces wirelength and power while improving the overall performance [55].

TABLE 1. Overview of learning-based methods used for wirelength optimization.

S. No	Paper & Year	Dataset	Design Stage	Data Source	Input Features	Target Label	Learning Paradigm	Algorithm	Evaluation Metrics
1	Chen et al. (2018) [48]	CAD contest & ISPD2006	Floorplan	Open-source macro placer (ic-cad2015_evaluate.pl)	16 features, including cell, net, pin density, macro locations	Routing congestion & HPWL	Supervised	ML-regression models, iteratively trained and evaluated	R squared (R^2), MAE
2	Lu et al. (2021) [51]	Multicore CPU (TSMC 28 nm)	Placement	Synthesized netlist using Synopsys design compiler, placement def from ICC2	Cell, net, pin density, Node features	Node embeddings for optimal placement	Unsupervised	Variational-GCN & Weighted K-means Clustering	Wirelength, Power, WNS/TNS
3	Xu et al. (2022) [52]	MCNC & GSRC	Floorplan	Circuit Netlist, floorplan information	Netlist graph, node attributes	negative HPWL	Reinforcement	GCN-RL, determines the location of circuit blocks	HPWL, Area, Runtime
4	Liu et al. (2022) [55]	ISPD2005, industrial designs	Placement	Netlist, floorplan information	Weighted adjacency matrix, macros, cell clusters, cell connectivity	Floorplan coordinates, HPWL	Unsupervised	C-spectral clustering with unsupervised loss function	HPWL, Runtime, Overflow
5	Yang et al. (2022) [56]	Industrial GPU/CPU designs	Placement	Netlist, placement reports of 5nm designs	Node features, Adjacency Matrix	Cell clustering for placement	Unsupervised	Graph clustering using PPA metrics as loss functions	Wirelength, WNS, TNS, Clock Power
6	Agnesina et al. (2022) [57]	Opencores, ISPD2012, RISC-V	Placement	Circuit Netlist, Placement parameters from Innovus	12 placement parameters, including eco max distance, max density, timing effort, clock gate aware	Optimal placement	Reinforcement	GNN & LSTM agent with attention, tunes parameter settings without supervised samples	HPWL, Runtime, Congestion
7	Amini et al. (2022) [58]	GSRC & MCNC	Floorplan	Netlist, placement reports	Block features, pin coordinates, hypergraph connectivity, CBL features (orientation, T-junction, rotation)	Block placement	Reinforcement	Hypergraph-GCN & dot product structure to handle large discrete action space	HPWL, Runtime

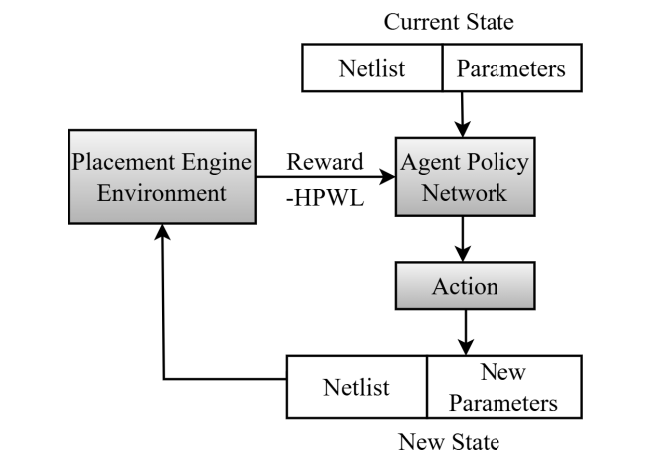


FIGURE 6. Workflow of DRL-based placement optimization, where the placement engine serves as environment and agent policy network learns to adjust parameters through actions [57].

imbalance. MSANet [62], a multi-scale feature extraction and fusion method for congestion hotspot prediction. Wang et al. proposed a GNN architecture based on Lattice

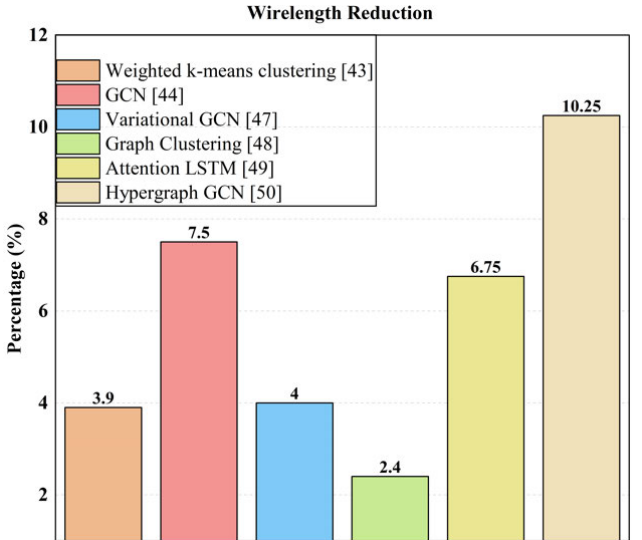


FIGURE 7. Wirelength reduction (%) across different learning-based models.

Hypergraphs (LH) for placement optimization [63]. This method enhances training accuracy by capturing netlist

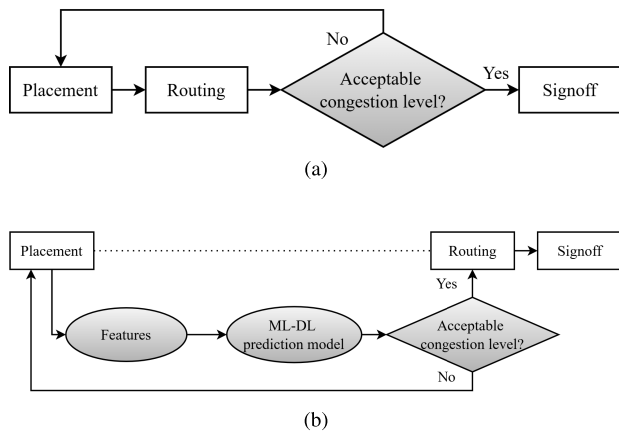


FIGURE 8. Comparison of RC prediction flows [59]: (a) Traditional flow without learning-based prediction (b) ML-DL flow integrating a prediction model.

topology and placement features, reducing routing congestion and wirelength.

GNNs are popular for modeling congestion because of their inherent ability to encode circuit connectivity effectively. Kirby et al. [64] introduced a DL-based Graph Attention Network (GAT) for congestion prediction in metal layers with cell attributes and pin information. The GAT consists of eight input layers and 16 hidden layers in the network architecture. GAT improves congestion prediction accuracy by 29% in all metal layers and by 75% in the lower metal layer. Wang et al. proposed a GNN architecture based on Lattice Hypergraphs (LH) for placement optimization [63]. This method enhances training accuracy by capturing netlist topology and placement features, thereby reducing routing congestion and wirelength. A cluster-level GNN model, ClusterNet [65], further refines congestion prediction by partitioning the netlist into groups of clusters and effectively captures netlist connectivity. Experimental results on OpenCore designs show that ClusterNet achieves an MAE 0.056 on unseen designs, outperforming GNN baselines and other clustering approaches. Recently, Peng et al. [66] presented the Steiner Tree Grid Segment GNN (STGSG) model that leverages Steiner tree-based GSegments instead of the traditional bounding boxes of multi-terminal nets. This approach enables a more accurate modeling of routing demand, particularly for large nets. The model achieved a 94.27% accuracy, a 76.36% F1-score and an 3.72% False Positive Rate (FPR), demonstrating its effectiveness in pre-routing congestion prediction.

Transformer-based models and hybrid approaches combine local and global feature extraction for more robust congestion prediction. The Regression Vision Transformer (R-ViT) [67] combines ViT and CNN-based upsampling for better fine-grained and global spatial feature extraction. It outperforms baseline learning-based models with more accurate congestion predictions. Cai et al. [59] proposed a lightweight Swin transformer-based RC prediction model with a Feature Pyramid Network (FPN) for efficient

multi-scale feature fusion. It uses shifted window attention for global feature extraction with a linear computational complexity. To address data sparsity, this model was pre-trained on the CircuitNet dataset and then fine-tuned for ISPD '15 benchmarks, thereby improving the cross-node generalization. In ISPD'15, this achieves an 11% SSIM (Structural Similarity Index Measure) improvement and a 10.8% NRMSE (Normalized Root Mean Square Error) reduction compared with LayNet with 2.74 M parameters and 2.59 G FLOPS.

LayNet [68], a multimodal congestion prediction, uses both Gcell and netlist connectivity information using a heterogeneous message passing mechanism and achieves up to 9.6% improvement over previous works. Gu et al. [69] proposed a dual (Early and Deep) Feature Fusion (EFF and DFF) technique using CNN and transformer layers for the routing congestion prediction task. It extracts grid-based features and netlist connectivity information using CNN and Hetero-GCN (Heterogeneous Graph Convolutional Network), respectively. EFF and DFF were employed with self- and cross-attention layers to refine multimodal features, and the output was cascaded to the decoder for accurate congestion heatmap predictions. The experimental results show higher prediction accuracy and RC reduction when integrated with the open-source placer DREAMPlace [70]. Figure 9 shows both the training flow for learning-based RC prediction and its optimization. The trained model was integrated into the RC optimization flow, where the design netlist was placed using DREAMplace, then congestion maps were predicted, and the outcomes were validated using Innovus eGR with reports of wirelength, vias, and congestion rates. Table 2 lists the learning-based models used for RC prediction, and Fig. 10 compares the different learning-based methods for RC prediction using NRMSE and SSIM. Models that employ advanced feature extraction and fusion mechanisms, such as transformer-based and GNN-transformers, exhibit lower error and higher structural similarity. This demonstrates the advantage of integrating local spatial details with the global contextual information.

C. DESIGN RULE CHECK (DRC) VIOLATION PREDICTION

As chip designs grow more complex, manually detecting and correcting hotspot violations in PnR becomes increasingly difficult and expensive. Learning-based models can predict potential DRC violations at an early stage and provide accurate solutions, thereby reducing iterations and time consumption in PnR, leading to high-quality and more efficient chip design. Figure 11 illustrates the ML-DL model workflow for DRC violation prediction. The ML-DL models are trained using features extracted from placement def files and labels from post-routing DRC reports. This training makes learning models predict DRC violations in the placement stage, providing necessary early feedback and minimizing the need for costly routing iterations. Huang et al. [71] integrated a

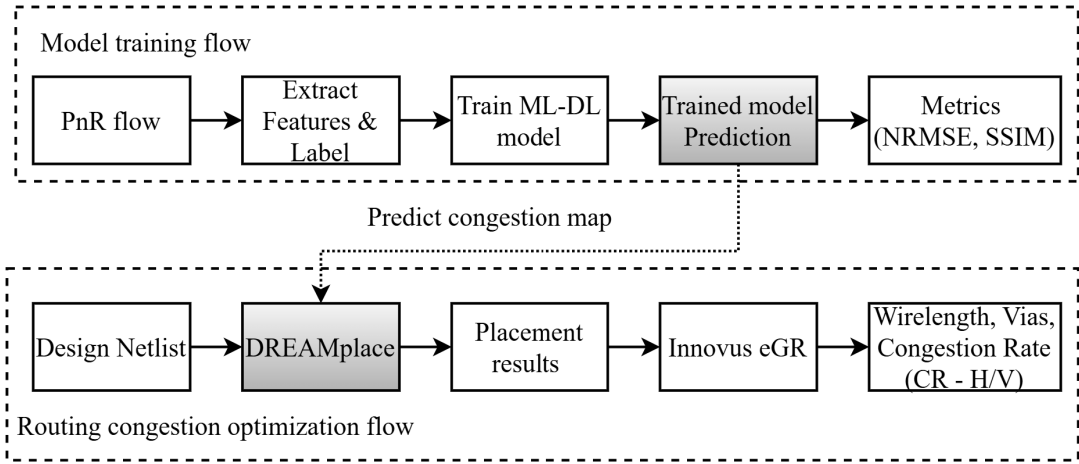


FIGURE 9. Learning-based RC prediction and optimization framework, integrating ML-DL congestion prediction model into the DREAMPlace [69].

TABLE 2. Summary of learning-based methods used for routing congestion prediction.

S. No	Paper & Year	Dataset	Design Stage	Data Source	Input Features	Target Label	Learning Paradigm	Algorithm	Evaluation Metrics
1	Chen et al. (2018) [48]	CAD contest & ISPD2006	Floorplan	Open-source macro placer (ic-cad2015_evaluate.pl)	16 features, including cell, net, pin density, macro locations	Routing congestion & HPWL	Supervised	ML-regression models together trained and evaluated	R squared (R^2), MAE
2	Kirby et al. (2019) [64]	Industrial designs	Synthesis	Design netlist, placement def, congestion reports	Netlist graph, cell features, pin count	per-cell detailed routing congestion	Supervised	GAT, predicts congestion in lower metal layers	Kendall Correlation Coefficient (KCC)
3	Wang et al. (2022) [63]	ISPD2011 & DAC2012	Placement	placement def file using DREAMplace, congestion reports	cell, net -density, pin count, heterogeneous graph	Routing demand, congestion map	Supervised	Lattice Hypergraph Neural Network (LHNN)	F1-score, Accuracy
4	Zheng et al. (2023) [68]	ISPD2015	Placement	Circuit Netlist, lef, def, congestion reports from Innovus	Macro region, RUDY, RUDYpin, heterogeneous graph	Horizontal & vertical congestion overflow map	Supervised	LayNet, Multi-scale vision transformer & contrastive learning	SSIM, NRMSE
5	Min et al. (2023) [65]	Opencore designs	Synthesis	Netlist using DC compiler, PnR from Innovus	Cell, net, pin - density, topological features, average fanout	cluster congestion value	Supervised	ClusterNet-GNN	MAE, R^2 , NRMSE, PCC, SCC, KCC
6	Li et al. (2024) [60]	CircuitNet	Placement	lef, def, congestion, DRC reports	Macro map, RUDY, pinRUDY	congestion, DRC	Supervised	Inception-boosted U-Net variant	NRMSE, SSIM, AUC
7	Chen et al. (2024) [61]	CircuitNet	Placement	design lef, def, congestion, DRC reports	Macro map, RUDY, pinRUDY	congestion, DRC	Supervised	Attention enhanced U-Net	NRMSE, SSIM, AUC
8	Liu et al. (2024) [62]	ISPD2015 & CircuitNet	Placement	placement def files, congestion report from Innovus	Macro region, RUDY, pinRUDY	Congestion heatmap	Supervised	MSANet, Multi-scale attention CNN	NRMSE, SSIM
9	Gu et al. (2024) [69]	ISPD2015	Placement	lef, def from DREAMplace, congestion reports	Macro map, cell density, RUDY, pinRUDY	routing congestion map	Supervised	CNN and transformer multimodal fusion	NRMSE, SSIM
10	Mo et al. (2024) [67]	CircuitNet	Placement	lef, def, congestion reports	Macro map, cell density, RUDY, pinRUDY	congestion map	Supervised	Regression-ViT	NRMSE, SSIM, ROC, PRC
11	Cai et al. (2025) [59]	CircuitNet & ISPD2015	Placement	lef, def, congestion reports from Innovus	Macro map, cell density, RUDY, pinRUDY	congestion map	Supervised	Swin Transformer	NRMSE, SSIM
12	Peng et al. (2025) [66]	Opencore designs	Pre-routing	placement def files	Cell, net, pin - density, G-segment features	Congestion overflow	Supervised	Steiner tree G-segment GNN	Accuracy, F1 score, False Positive Rate (FPR)

classical ML approach into the placement flow, using the *LibSVM* – 3.24 SVM (Support Vector Machine) multi-class

classifier, which is trained on placement and routing results to refine the placement and predict DRC violations.

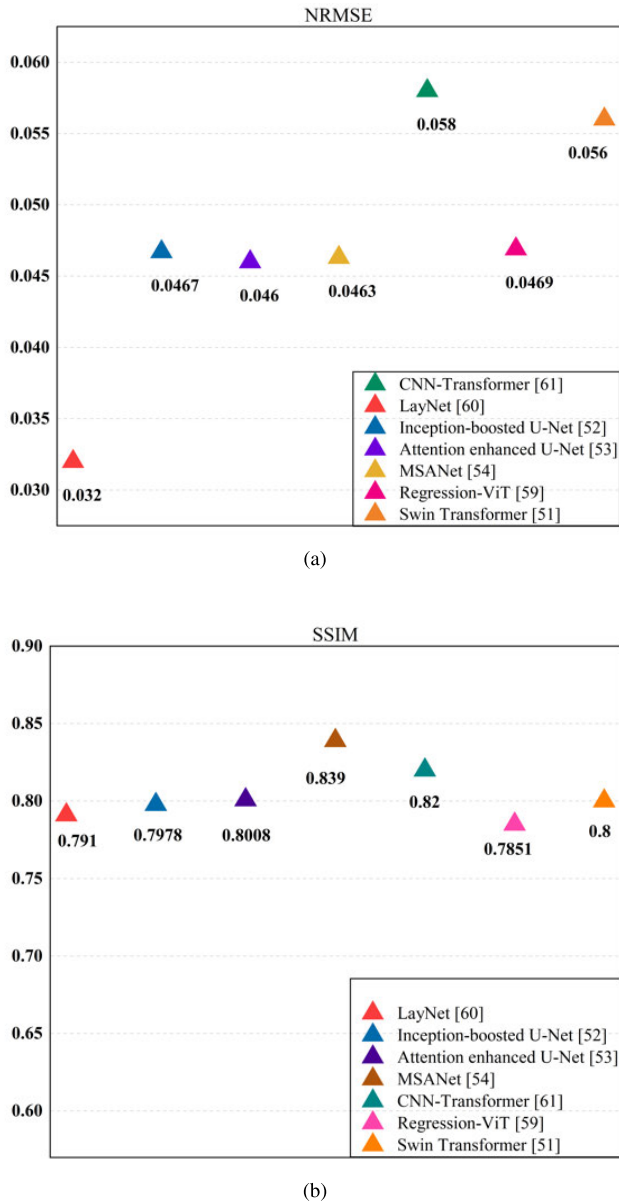


FIGURE 10. Performance comparison of learning-based RC prediction methods [59]: (a) NRMSE (b) SSIM.

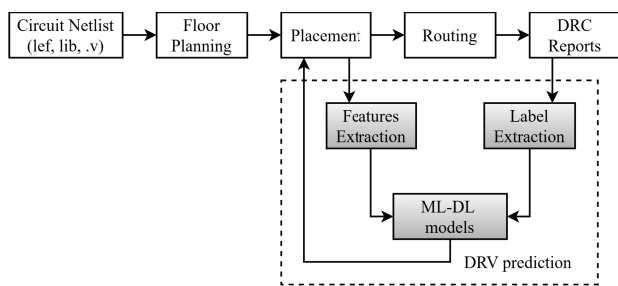


FIGURE 11. DRC violation prediction flow using learning-based methods.

CNNs have been extensively used for DRC violation prediction. RouteNet [72] employed a pretrained ResNet and Fully Convolutional Network (FCN) architecture to predict DRC hotspots at macro placement. Its performance is superior

compared to that of an SVM and logistic regression. Similarly, Huang et al. [73] integrated a CNN-based routability predictor into a macro placer, where reliable macro placement with minimal DRV was achieved through an optimal Simulated Annealing (SA) approach. However, the dependence on the SA optimizer raises concerns regarding its scalability and convergence. J-Net is a further extension of the CNN model that employs pin and post-route information [74]. It improves the True Positive Rate (TPR) on 12 industrial designs at the 7nm technology node but causes high computation cost due to the large pixel pin pattern images.

Graph-based models have been explored to capture placement and connectivity features, providing improved accuracy and generalization over violation prediction. Chen et al. introduced an inductive-based model, GraphSAGE, for DRV prediction [75]. GraphSAGE aggregates the vertex information for the embeddings of unknown vertices, giving better DRV classification even with small datasets. This model gives the highest prediction (0.53) of DRV for all cases, such as $shorts < 100$, $100 < shorts < 500$ and $shorts > 500$ when compared to the other learning models, RUSBoost (0.11) and GCN (0.42). In [76], a hybrid Pin accessibility-aware GNN and U-Net (PGNN) model is implemented, which combines grid-based features and pin proximity graphs to generate binary hotspot maps. This model outperforms the existing ML models, J-net [74] and RouteNet [72], with less inference time, provides accurate predictions of DRC hotspots, and significantly improves the F1 score over J-net, but causes high computational cost due to the dual-model structure.

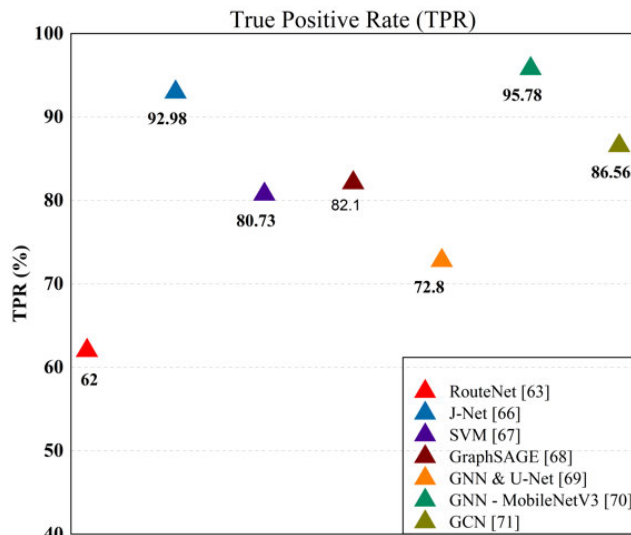
Recent studies have combined multiple architectures to improve predictive performance. Lin et al. [77] developed a serial model combining a GCN with CNN-based MobileNetV3-small to capture graph-structured placement features and spatial refinement. To improve the robustness, a sliding window expansion technique was employed to incorporate neighboring tile information. This method achieves an AUC of 98.35%, a TPR of 95.78%, and an FPR of 4.17% demonstrating higher accuracy and better generalization across unseen designs. In another work [78], Lin et al. used GCN with an adaptive adjacency matrix, in which it models routing-aware relationships based on bounding-box overlaps. Weighted binary cross entropy loss was used to handle the class imbalance. The model outperformed other baseline models with 87.69% accuracy, 86.56% TPR, and 9.62% FPR. Table 3 compares the recent learning-based approaches used for DRC violation prediction. Figure 12 represents the TPR values of different ML-DL models. While CNN-based approaches provide strong baselines, the GNN and hybrid models achieve higher accuracy and reliability in identifying DRC violations.

D. TIMING PREDICTION AND OPTIMIZATION

Timing is typically evaluated using slack, which is the difference between required and actual arrival times. A positive slack indicates that timing constraints have been met, and negative slack signifies failure to meet the timing require-

TABLE 3. Learning-based methods used for DRC prediction.

S. No	Paper & Year	Dataset	Design Stage	Data Source	Input Features	Target Label	Learning Paradigm	Algorithm	Evaluation Metrics
1	Xie et al. (2018) [72]	ISPD2015	Placement	Design lef, def, DRC reports from Cadence Encounter	Macro region, cell density, RUDY	DRV count level, DRC hotspot map	Supervised	RouteNet - Fully Convolutional Network (FCN)	Accuracy, TPR, FPR
2	Huang et al. (2019) [73]	ISPD2015	Floorplan	Design lef, def, DRC reports from Cadence Encounter	Macro density, Pin density, net-macro connectivity	DRVs	Supervised	CNN - VGG16	Best rank in top % placement, total wirelength
3	Liang et al. (2021) [74]	Industrial designs	Placement	Circuit lef, def, DRC reports	Cell density, macros, pin configuration features	DRC	Supervised	J-Net	Accuracy, Precision, Recall, F1-score, False Positive Rate (FPR)
4	Huang et al. (2021) [71]	ISPD2014	Placement	Circuit netlist, lef, def files	Cell density, local & global nets, clock pins, non-default routing nets	Short violations	Supervised	Support Vector Machine (SVM)	TPR, Specificity, Accuracy, Matthews Correlation Coefficient (MCC)
5	Chen et al. (2021) [75]	ISPD2015	Placement	Netlist, def, DRC reports	Cell, macro, pin, blockage - density, local & global net, vias, relative position	Short violations	Supervised	GraphSAGE	Accuracy, TPR, TNR, precision, F1-score
6	Baek et al. (2022) [76]	OpenCore designs	Placement	PnR database using Synopsys ICC2 compiler	Node & edge features, pin density, local & global nets, RUDY, H/V routing capacity	DRC hotspot	Supervised	GNN & U-Net	Accuracy, precision, recall, FPR, F1-score
7	Lin et al. (2024) [77]	ISPD2015	Placement	lef, def files, DRC reports	Macro, cell, pin - density, local & global nets, RUDY features	DRC hotspot	Supervised	GNN - MobileNetV3	TPR, FPR, AUC
8	Lin et al. (2025) [78]	ISPD2015	Placement	lef, def files, DRC reports	Macro, cell, pin, blockage - density, RUDY, RUDYpin, routing resources	DRC hotspot	Supervised	Graph Convolutional Network (GCN)	Accuracy, TPR, FPR, AUC


FIGURE 12. True Positive Rate (TPR) comparison of learning-based models for DRC violation prediction.

ments. Total Negative Slack (TNS) and Worst Negative Slack (WNS) are the metrics used to check for timing problems. TNS represents the sum of all negative slacks across violating paths, whereas WNS pinpoints a single path with a severe timing issue. Bai et al. proposed ML-based models for timing prediction at the floorplan stage, eliminating the need for PnR

and Static Timing Analysis (STA) [79]. A hybrid ensemble model was developed that combines a Back Propagation Neural Network (BPNN) and SVM predictions with AdaBoost weighted output. On 500 samples of Opencore IP designs, this hybrid model achieved an MSE of 3.1 compared to BPNN (3.7) and SVM (3.4). This approach reduced the prediction runtime 3h of the STA to approximately 1 minute, enabling fast slack estimation and significantly reducing the number of design iterations. Guo et al. implemented a timing engine-inspired GNN model for pre-routing slack prediction [80]. It consists of two stages: a net embedding model that extracts routing-related net delays from placement data and a delay propagation technique to propagate timing information for estimating arrival time and slew prediction. Evaluations of 21 open-source benchmarks using OpenRoad showed that the proposed model achieved higher accuracy and better generalization than traditional STA.

A framework for physical and timing prediction before the detailed placement stage was proposed in [81], which provides promising solutions for candidate gate sizing, buffer insertion, and path groups for violated paths using a GNN. Figure 13 illustrates the optimization guidance framework that enhances placement quality by integrating physical and timing prediction. Using a synthesized netlist, an initial layout was generated and analyzed using a GNN-based prediction module. It employs specialized GAT modules of GAT^s for gate sizing, GAT^b for buffer

insertion, and GAT^t for group path optimization. These models provide timing guidance for improving layout quality before detailed placement. This process yields an optimized layout with better PPA characteristics than those of traditional flows. Experimental results demonstrate an average of 24.21 % TNS, 31.32 % in the Number of Violating Paths (NVP), and 2.33 % WL reduction compared with traditional models. PRO-TIME [82] is a multimodal DL approach for pre-routing timing prediction. It integrates an endpoint-wise netlist and layout information extracted using GNN and U-Net architectures for fast and accurate timing estimation. It runs at an impressive rate of 3297 times faster compared to a commercial tool with a 10 % R^2 score loss.

A novel multimodal framework is proposed by He et al. that explicitly incorporates Power Delivery Network (PDN) information [83]. This study integrates the PDN, layout, and netlist features to predict the endpoint arrival time. It combines U-Net with a Swin Transformer for PDN or layout features and a heterogeneous GNN model for netlist features processing along with auxiliary and contrastive learning. The loss function used was a combination of timing, congestion, IR drop, and contrastive PDN-aware loss for generalization. Experiments on Opencore benchmarks show that the proposed model achieved an average R^2 of 0.9438 compared to unimodal (0.6993) and bimodal (0.8643) baselines.

Lim et al. introduced an unsupervised GNN for timing-driven detailed placement, which integrated timing-driven objectives in the optimization process [84]. The loss function balances the overlap, alignment, timing arc, and boundary constraints with hyperparameters tuned via Bayesian optimization. In open-source designs, this method reduces TNS 5.5% compared to OpenDP [85] while maintaining comparable WNS, demonstrating effective timing-driven placement improvements. An effective timing prediction algorithm for the global routing stage, the Kolmogorov-Arnold Network U-Net (KAN-U-Net), was utilized in [86]. KAN performs non-linear netlist relationships, and U-Net captures spatial features from placement and congestion maps for slack regression. This model achieved an R^2 of 0.956 and when integrated into global routing optimization, it reduced TNS by 46.2% and the number of violating paths by 39.7%. Lin et al. presented a multitask GNN model for the simultaneous prediction of endpoint slack and net delay, yielding more accurate and consistent results [87]. The model is trained with slack MSE and weighted net delay MSE loss, using the Adam optimizer with ReLU activations. The experimental results on the DAC2022 designs indicate that the model outperforms the baselines by achieving R^2 of 0.9058, along with reductions in WNS and TNS.

E2ESlack is an end-to-end GNN-based framework that integrates both Arrival Time (AT) and Required Arrival Time (RAT) estimation to enable reliable prediction of slack, TNS, and WNS before routing [88]. First, a TimingParser

converts files from OpenLane flow into a heterogeneous graph representation. This graph is used by an AT prediction model, a GNN model trained to predict arrival times and related pin-level features. An RAT estimation algorithm propagates the required times with clock definitions and constraints from primary outputs back to primary inputs. Experimental results showed that E2E slack significantly outperformed both pre-routing STA and TimingPredict, reducing the MAE in RAT estimation by 57%, and producing TNS and WNS results comparable to the post-routing STA with up to 23 times faster runtime. A multitask graph learning algorithm that concurrently predicts wirelength, wire delay, and cell delay during placement [89]. The circuit was modeled as a graph with cells as nodes and nets as edges, enriched with placement and timing features. The results indicated improved accuracy and generalization over single-task models with strong transferability across technology nodes and unseen designs, highlighting the benefit of cross-task parameter sharing.

Table 4 presents a comprehensive summary of recent learning-based approaches for timing prediction and optimization. The coefficient of determination (R^2) measures how well a regression model's predictions align with actual data. Figure 14 represents the comparison of R^2 values for various learning-based algorithms used in delay prediction. A higher R^2 value indicates better model fitting and predictive accuracy.

E. IR DROP AND POWER ESTIMATION

IR drop analysis measures the voltage drop across the on-chip Power Delivery Network (PDN), and it is critical for advanced technology nodes due to reduced supply voltages and high current densities. Similarly, power estimation is essential to ensure that the PDN can deliver sufficient current to all regions of the chip without violating Electro-Migration (EM) limits. Emerging learning-based models predict IR drop hotspots and power consumption early in the PnR stages, enabling fast and accurate PDN optimization. Chhabria et al. [90] proposed two optimized neural networks, Multi-Layer Perceptron (MLP) and LeNet-based CNN at the floorplan and placement stage with pre-designed templates to build an efficient PDN that satisfies safe EM and IR drop. The MLP consists of a fully connected input layer with 2295 nodes, followed by three hidden layers of nodes (256, 128, 64), and an output layer with eight nodes representing the 8 optimal templates (classes). Using Innovus on a RISC-V core in 16FF technology, this work added 3 percent more routing resources in congested areas that may assist timing closure in advanced chips.

PowerNet [91] advances dynamic IR drop estimation by integrating physically interpretable preprocessing with a Max-CNN architecture, enabling it to efficiently capture both spatial correlations and transient behavior. This approach achieves up to 30 $\times$ speedup over commercial tools and 9% more accuracy than CNN/XGBoost baselines across multiple sub-10nm designs. In [92], a model was designed

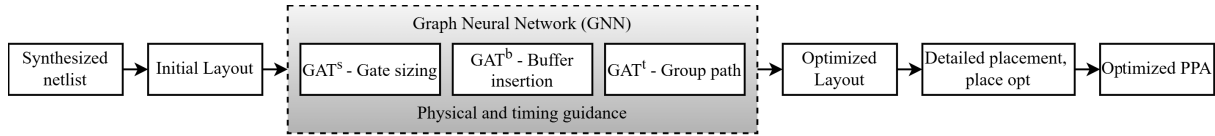


FIGURE 13. Optimization guidance framework utilizing physical and timing prediction for better placement quality [81].

TABLE 4. Summary of learning-based methods used for timing prediction and optimization.

S. No	Paper & Year	Dataset	Design Stage	Data Source	Input Features	Target Label	Learning Paradigm	Methodology	Evaluation Metrics
1	Bai et al. (2018) [79]	OpenCores IPs	Floorplan	Netlist, sdc, floorplan def from Synopsys ICC2 compiler	Clock period, Max transition, max fan out, cell height, width, pitch, min distance	Slack time	Supervised	Combines BPNN and SVM with AdaBoost Ensemble model to explore non-linear feature extraction	MSE, Confusion Matrix
2	Guo et al. (2022) [80]	Open-source designs	Placement	Circuit Netlist, lef, def files from OpenRoad	Pin and edge features	Arrival time, Slack, Net delay, Cell delay	Supervised	GNN, net embedding and delay propagation	R squared (R^2), Runtime
3	Cao et al. (2025) [81]	OpenCore designs	Placement	Netlist, def, timing reports	Worst input & output slack, max input & output skew, delay, cell area, max distance, fan in & fan out	Gate sizing, buffer insertion, violation paths	Supervised	Graph Attention Network (GAT) - GAT ^a , GAT ^b , GAT ^c together performs multitask prediction	F1-score, WNS, TNS, Number of violating paths (NVP), Wirelength
4	Wang et al. (2025) [82]	RISC-V designs	Placement	Netlist, def, timing reports from Innovus	Macro, cell density, RUDY, net distance, cell driving strength, pin capacitance	Endpoint arrival time	Supervised	U-Net, extracts spatial features & customized attention-based GNN for slack regression	R squared (R^2)
5	He et al. (2025) [83]	Opencore designs	Placement	Netlist, def, STA results	Cell, pin - density, RUDY, power/ground stripes, vias, fan out, capacitance, power maps	End-point arrival time	Supervised	U-Net & Swin Transformer for local-global context, learns path-level timing	R^2 , RMSE
6	Lim et al. (2025) [84]	Open-source designs	Placement	Netlist, lef, def, STA files	Cell features, X-Y coordinates, driving strength	Optimized X-Y coordinates with reduced slack	Unsupervised	Graph Convolutional Network (GCN) optimizes directly using custom loss function	TNS, WNS
7	Liu et al. (2025) [86]	Industrial designs	Global routing	Netlist, lef, def, timing lib	Driver slew, parasitic RC, congestion map	Wire delay	Supervised	Kolmogorov-Arnold Network (KAN), uses spline functions	TNS, WNS
8	Lin et al. (2025) [87]	DAC2022	Placement	Netlist, placement def, timing reports using OpenRoad	Macro region, RUDY, cell density, fan-in/ fan-out, load capacitance, graph features	Slack, Net delay	Supervised	Heterogeneous GNN, captures both logical and physical features	R squared (R^2)
9	Bodhe et al. (2025) [88]	OpenLane designs	Placement	Netlist, lib, lef, placement def, sdf files using OpenLane	Pin node, net edge and cell edge features	Required Arrival Time (RAT), TNS, WNS	Supervised	End-to-end GNN with Timing parser for graph construction	R squared (R^2), MAE
10	Xing et al. (2025) [89]	Opencore designs, ISCAS89	Placement	Placement def, cell lib, timing reports from OpenSTA	Node, edge, path features, graph netlist	Wirelength, wire delay, cell delay	Supervised	Multitask graph learning (MTGL), parameter sharing & residual connections	R squared (R^2), NMSE

for instance-level accuracy, leveraging the U-Net architecture augmented with 3D convolutional encoding to integrate per-instance power and physical features. This enables the model to capture the sparse and bursty nature of switching activity across different time intervals. Another notable contribution is the regression-coefficient decoding mechanism, which generates interpretable coefficient maps that scale with instance-level features, thus improving transferability across unseen designs. In [93], a multitask CNN model with EfficientNet-B0 was utilized to predict dynamic IR drop and routing congestion. This achieves high accuracy, delivers up to $29\times$ speedup, and reduces routing resource usage by

approximately 13%, with minimal degradation in IR drop performance.

CFIRSTNET [94] significantly enhances the accuracy and efficiency of static IR drop estimation by fusing image-based and netlist-derived features within a customized CNN that captures both the geometric and electrical characteristics of the PDN. Figure 15 illustrates the overall workflow of CFIRSTNET to generate the predicted IR drop map. In [95], ICdaIR employed a CNN-U-Net regression model for distributed-aware static IR drop prediction and a random forest classifier to identify hotspot subregions. The MAE of ICdaIR decreased by 50.5% and 14.4% compared with

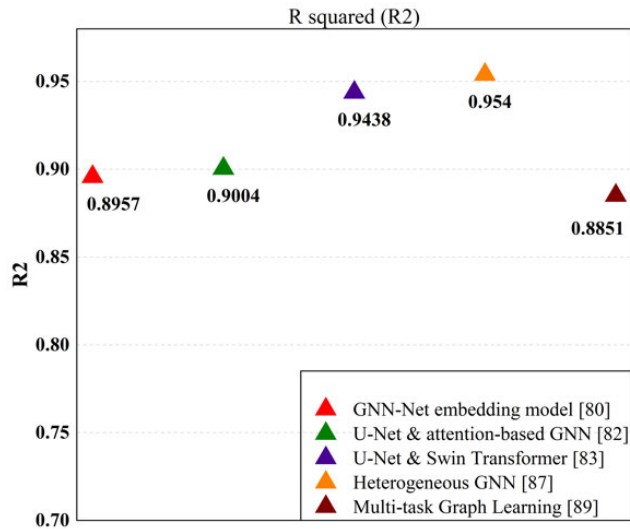


FIGURE 14. R² comparison of learning-based models for delay prediction.

the state-of-the-art method and models without classifiers, respectively. Mondal et al. introduced an ANN-based prediction model for static and dynamic IR drop in PDNs [96]. A feed-forward ANN with four hidden layers (130 – 64 – 32 – 16 neurons) was designed to predict static IR drop values based on placement-level features. For dynamic IR drop prediction, a bagged ensemble of ANNs was trained on the CircuitNet dataset to improve robustness and generalization.

An enhanced TransU-Net model for static IR drop prediction that modifies the U-Net architecture using a transformer encoder was proposed in [97]. The hybrid design effectively captures the spatial complexity of IR drop behavior using convolutional layers for local feature extraction and transformers for global context modeling, thereby achieving accurate hotspot localization and lower error across test circuits with reduced runtime. LaRED [98] is an encoder-decoder architecture structured with three core modules: a rebuilder that uses deformable convolutions to organize uneven raw input features, an encoder that captures both local and global layout information through hybrid downsampling and attention fusion, and a decoder that reconstructs fine-grained IR drop maps using cascaded upsampling and dual-skip connections. Experimental results demonstrate that LaRED achieves a 34.6% – 42.6% MAPE improvement over baselines along with a reduction in hotspot misprediction. Lan et al. proposed a high-speed, robust, and cross-node transferable model that predicts a static IR drop with minimal training data requirements and high accuracy for new technology unseen designs [99]. It combines a CNN (U-Net) and RNN (GRU) to capture both the spatial dependencies of the PDN layout and the sequential relationship across different metal layers. The model achieved an average 10.4% error rate on unseen circuits with limited fine-tuning samples, addressing the key limitations of many earlier studies.

Table 5 outlines the progression of ML-DL models developed for IR drop and power estimation, evolving from simple MLP and CNN models towards hybrid architectures of CNN-transformer and CNN-RNN that balance accuracy, interpretability, and cross-design generalization.

F. AI ACCELERATORS

Recent advancements in neural network hardware accelerators have enabled the efficient execution of deep learning models through specialized architectures on CPU, GPU, ASIC, and FPGA-based platforms [100]. Modern hardware accelerators such as multithreaded CPUs or GPUs are leveraged together with ML-DL models to speed up the PnR process [3], [101].

DREAMPlace, a GPU-accelerated deep learning toolkit was proposed by Lin et al. [70] for optimizing placement. Figure 16 shows the DREAMPlace toolkit flow and its software implementation. The goal of DREAMPlace is to reduce the cell density and minimize the wirelength. It is implemented in Python using PyTorch and C++/CUDA with specialized kernels for wirelength and density computations, offering high-speed placement solutions for both the GPU and CPU platforms. Experimental results on the ISPD2005 benchmark [102] and industrial designs demonstrate that DREAMPlace achieves 38x speed and 47x speed in global placement on a GPU with comparable quality to the multithreaded CPU-based RePlace [20], thus directing EDA solutions towards AI accelerator-based implementations.

Liu et al. [103] enhanced the DREAMPlace toolkit by integrating a Fully Convolutional Network (FCN) model for efficient and accurate congestion map prediction, considering congestion penalty along with wirelength and density in the placement cost function. This model achieves a 9.05% reduction in congestion and a 5.3% reduction in wirelength compared to baseline DREAMPlace. Lu et al. [104] proposed a placement optimization method using generative adversarial learning (DREAM-GAN) with CNN and GNN discriminators to upgrade the DREAMPlace toolkit. In addition to wirelength and density computations, DREAM-GAN computes similarity loss and adversarial learning, enabling better quality placements. With five commercial CPUs and open-core designs using the 28nm technology node, DREAM-GAN shows PPA improvements of 8.3% wirelength and 7.4% power reduction in the placement and post-routing stages.

Furthermore, Guo et al. [105] introduced a DNN-based differentiable, timing-driven placement engine. In addition to the wirelength and density gradient computation in the DREAMPlace toolkit, this model added the WNS/TNS gradient for timing analysis. Implementation results show improved performance and efficiency, achieving 32.7% WNS and 59.1% TNS improvement over prior works [70], [106]. Han et al. [107] developed a GPU-accelerated global router to improve the parallelism and simplify routing. This achieves a 2.5 – 3.9× runtime improvement with a 2.5% increase in wirelength over NCTUgr 2.0 [42]. GAMER [108] is

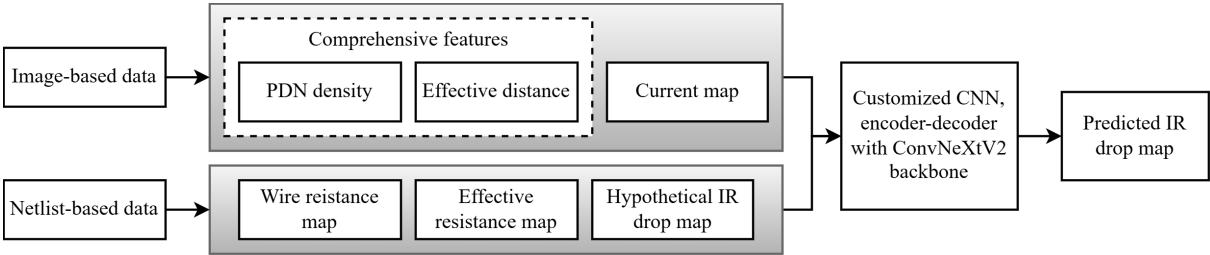


FIGURE 15. Overview of CFIRSTNET [94]: Image-based and Netlist-based data are integrated and processed through a customized CNN to predict IR drop map.

TABLE 5. Learning-based frameworks applied to IR drop and power analysis.

S. No	Paper & Year	Dataset	Design Stage	Data Source	Input Features	Target Label	Learning Paradigm	Algorithm	Evaluation Metrics
1	Chhabria et al. (2020) [90]	RISC-V designs	Floorplan & Placement	post-placement current & congestion maps	Current distribution, congestion per region	PDN template ID (8 classes)	Supervised	MLP & LeNet-CNN	IR drop, EM
2	Xie et al. (2023) [91]	6 industrial designs - 10nm	Post-CTS	Placement def, power analysis reports, STA reports	Internal power, switching power, leakage power, arrival time, toggle rate	Tile-level IR drop map	Supervised	Max-CNN, a maximum selection mechanism	ROC-AUC, MSE, Kernall rank coefficient
3	Chhabria et al. (2021) [92]	Industrial designs	Signoff	VCD simulation traces, power/voltage maps	Per-cycle switching activity, localized power, spatial-temporal toggle features	Dynamic IR drop	Supervised	3D-CNN, U-Net with classification head	RMSE, MAE
4	Huang et al. (2023) [93]	Industrial designs	Placement	Placement results, IR drop labels, IR drop and congestion labels	Cell-based, pin-based, local & global features	Dynamic IR drop, congestion	Supervised	EfficientNet-B0, multitask learning	MAE, MAPE, MaxE, NRMSE, CC
5	Liu et al. (2024) [94]	BeGAN benchmarks	Post-placement	Public dataset files from OpenNPDN, OpenRoad - ICCAD2023	Image & netlist-based maps - current, PDN density, effective distance to voltage source, wire resistance, resistive distance	Static IR drop map	Supervised	Customized CNN, encoder-decoder with ConvNeXtV2 backbone	MAE, F1-score
6	Li et al. (2024) [95]	CircuitNet	Post-placement	Design left, placement def, power analysis report	Equivalent resistance, minimum resistance, leakage power, internal power, total power	Static IR drop map	Supervised	Random forest classifier & CNN-U-Net prediction model	MAE, F1 score
7	Mondal et al. (2025) [96]	ISPD2019, Circuit-Net	Routing	Circuit netlist, def, power map from Voltus	No. of sinks, HPWL, wirelength, resistance, internal power, switching power, toggle rate, total power, leakage power, load capacitance	Static and dynamic IR drop	Supervised	Feed-forward and Bagged Ensemble ANN - dual ANN framework	Correlation coefficient, MAE, MSE, RMSE, R ²
8	Zuo et al. (2025) [97]	CircuitNet-28	Routing	Design left, def, power map, congestion map	multi-channel power features, Macro region, RUDY	Static IR drop, congestion map	Supervised	Enhanced TransU-Net uses transformer encoder in the U-Net	MAE, MSE, NRMSE, SSIM
9	Yu et al. (2025) [98]	CircuitNet-14	Signoff	lef, def, power maps	Power map, effective resistance map, minimum path resistance map	Static IR drop distribution map	Supervised	LaRED, Layout preserving Re-builder-ENcoder-Decoder with deformable conv & attention fusion	MAPE, MAE, Max Absolute Error (MaAE), correlation coefficient
10	Lan et al. (2025) [99]	BeGAN benchmarks	Signoff	Public dataset files from OpenNPDN, OpenRoad - ICCAD2023	Metal layers samples, current maps	Static IR drop map	Supervised	GRU-U-Net, Gated Recurrent Unit (GRU) arrays	MAE, NRMSE, CC

a GPU-accelerated maze routing framework that employs alternating wavefront sweep operations to explore multiple

routing paths simultaneously, leading to a significant reduction in runtime compared to the CPU-based router [109].

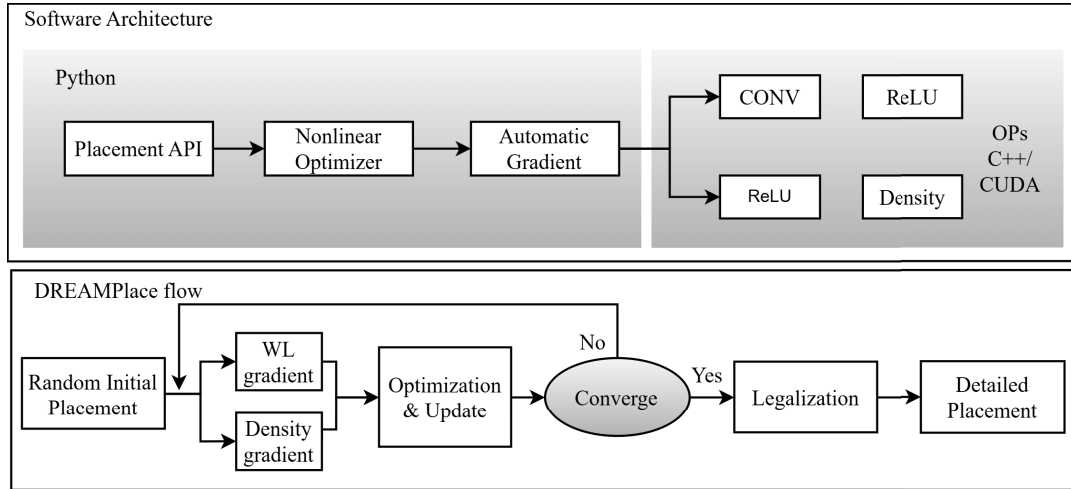


FIGURE 16. Overview of DREAMPlace, a GPU-accelerated DL Toolkit for VLSI placement [70].

Placement algorithms have seen the deep integration of AI and GPU computing, leading to substantial runtime improvements and better optimization quality. While routing frameworks have traditionally relied on GPU-parallelized methods, emerging research is moving towards AI-accelerated routing by integrating AI-based learning models to improve routing efficiency and quality. These developments indicate a clear trend towards unified, AI-driven, and fully accelerated PnR solutions.

IV. MODEL TRAINING AND PERFORMANCE

The performance of ML-DL models depends on both the architectural design and training setup, which directly affect the convergence and prediction accuracy. This section summarizes the widely adopted training and quality standards, including the loss functions, optimizers, and evaluation metrics.

A. LOSS FUNCTIONS

In AI models, choosing a proper loss function is important because it directly influences model training. Below are some commonly used loss functions for prediction and classification tasks [110].

1) MEAN SQUARED ERROR (MSE)

Also known as L_2 loss or quadratic loss, mainly used for regression models, shown in equation (1).

$$MSE(y, \hat{y}) = \frac{1}{M \times N} \|\hat{y} - y\|_2^2 \quad (1)$$

where $M \times N$ is the normalization factor, represents the height & width of an input image and $(y, \hat{y})$ is the actual and predicted values, respectively.

2) MEAN ABSOLUTE ERROR (MAE)

Also known as L_1 loss. It is the absolute difference between the actual and predicted values, shown in equation (2)

$$MAE(y, \hat{y}) = \frac{1}{M \times N} |\hat{y} - y| \quad (2)$$

3) BINARY CROSS ENTROPY LOSS (BCE)

Commonly used for binary classification tasks. It is the difference between predicted probabilities and actual labels, given in equation (3)

$$BCE(y, \hat{y}) = -\frac{1}{M \times N} [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \quad (3)$$

4) MEAN SQUARED LOGARITHMIC ERROR (MSLE)

Similar to γ_{mse} , it first applies logarithmic and then compute squared differences between actual and predicted values as shown in equation (4).

$$MSLE(y, \hat{y}) = \frac{1}{M \times N} (\log(1 + y) - \log(1 + \hat{y}))^2 \quad (4)$$

5) FOCAL LOSS

It is more efficient and robust with imbalanced datasets. The equation (5) provides the focal loss.

$$Focal(y, \hat{y}) = -\alpha (1 - p_t)^\gamma \log(p_t) \quad (5)$$

where,

$$p_t = \begin{cases} \hat{y}, & \text{if } y = 1 \\ 1 - \hat{y}, & \text{if } y = 0 \end{cases}$$

Here, α is the weight function, and γ is a hyperparameter.

6) HINGE LOSS

It is used in SVMs to train models by maximizing the margin between classes to improve robustness.

$$H_s(z) = \max\{0, s - z\} \quad (6)$$

where s is a constant, representing margins, and z denotes the model output.

7) HUBER LOSS

It combines γ_{mse} and γ_{mae} loss, making it less sensitive to outliers. For a predicted output (y_{hat}) and true label (y) with

threshold $\delta > 0$,

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta, \\ \delta \cdot \left(|y - \hat{y}| - \frac{1}{2}\delta\right) & \text{if } |y - \hat{y}| > \delta. \end{cases}$$

B. OPTIMIZERS

During training, the optimization algorithms adjust the weights and biases (model parameters) of the neural network to minimize the loss function. Proper selection of an optimizer is crucial in the modern deep learning training process [111].

Stochastic Gradient Descent (SGD) is a basic optimization algorithm that updates the model parameters using the gradient of the loss function, guiding the model towards minimal loss. Another widely used variant includes SGD with momentum and SGD with Nesterov acceleration, which provides smooth updates and fast convergence. Root Mean Square Propagation (RMSProp) is an adaptive optimizer that tunes the learning rate for each parameter based on the moving average of squared gradients to improve stability. Adaptive Moment Estimation (Adam) integrates momentum with adaptive learning rates and is commonly used to provide efficient and stable convergence. Adam with weight decay (AdamW), a variant of Adam that decouples weight decay from the gradient-based update, helps improve generalization and mitigate overfitting issues.

These optimizers ensure the effective and stable training of deep learning models across a wide range of applications.

C. EVALUATION METRICS

The performance of the model can be evaluated using the following metrics based on specific tasks [112].

1) ACCURACY

Accuracy defines the percent of correctly classified instances. This is not suitable for unbalanced datasets.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

where TP represents True Positives, TN denotes True Negatives, FP is the False Positives, and FN signifies False Negative samples.

2) TPR

The True Positive Rate (TPR), also referred to as Sensitivity or Recall, provides the percentage of true instances correctly identified.

$$\text{TPR} = \frac{TP}{TP + FN} \quad (8)$$

3) FPR

The False Positive Rate (FPR) indicates the percentage of false instances mistakenly classified as true positives.

$$\text{FPR} = \frac{FP}{FP + TN} \quad (9)$$

4) PRECISION

It measures the correctly predicted positive instances among all instances predicted as positives.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (10)$$

5) ROC CURVE

The Receiver Operating Characteristics (ROC) curve visually represents the trade-off between TPR and FPR.

The area below the ROC curve is the AUC score (Area Under the Curve), where a higher AUC indicates a better classifier performance.

6) F1-SCORE

F1-score helps in evaluating the model's performance by combining precision and recall into a single measure.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (11)$$

The aforementioned metrics are typically derived from the confusion matrix [113], which is employed for classification problems.

7) NRMSE

Normalized Mean Square Error (NRMSE), using min-max normalization, quantifies the pixel-wise difference between the predicted output and the ground truth. A lower value indicates better model performance.

$$\text{NRMSE} = \frac{1}{y_{\max} - y_{\min}} \sqrt{\frac{\sum_{m=1}^h \sum_{n=1}^w (y_{(m,n)} - \hat{y}_{(m,n)})^2}{h \times w}} \quad (12)$$

where (h, w) is the image size and, $y_{(m,n)}, \hat{y}_{(m,n)}$ represents the actual and predicted output values, respectively.

8) SSIM

The Structural Similarity Index Measure (SSIM) is used to assess the structural similarity between actual and predicted values. The higher SSIM value indicates greater model accuracy.

$$\text{SSIM} = \frac{(2\mu_{\hat{y}}\mu_y + C_1)(2\sigma_{\hat{y},y} + C_2)}{(\mu_{\hat{y}}^2 + \mu_y^2 + C_1)(\sigma_{\hat{y}}^2 + \sigma_y^2 + C_2)} \quad (13)$$

where $(\mu_{\hat{y}}, \mu_y)$, $(\sigma_{\hat{y}}, \sigma_y)$, $\sigma_{\hat{y},y}$ are the mean, variance and covariance of the predicted and actual values, respectively.

V. OPEN-SOURCE BENCHMARK SUITE

Open-source benchmarks from industrial ASIC designs have been made available to researchers to provide better PnR optimization. Table 6 summarizes the available benchmark designs and datasets for research on PnR automation and optimization.

Various designs from the ISPD placement contest [114], [115], [116], [117], [118], ICCAD CAD contest [119], [120], [121], [122], [123], DAC system design contest [124],

TABLE 6. Open-source benchmark suites and datasets for PnR automation research.

S.No	Benchmark designs	Overview	Task
1	ISPD [114]	A global academic and industrial design automation contest, focuses on physical design automation problems ISPD'19 [115] - 10 designs of detailed placement (45nm tech node) ISPD'15 [116] - 14 Industrial designs of blockage-aware placement (65nm and 28nm tech node) ISPD'11 [117] - 8 superblue designs of routability-driven placement ISPD'05 - 6 design samples for placement problem	Wirelength, Routing Congestion & DRC violation
2	ICCAD [119]	A contest in EDA provides industrial benchmark circuits and real-world problems to advance innovation and research ICCAD'23 [120] - It contains design samples of multiple technology nodes (ASAP7, Nangate45, Skywater130) ICCAD'19 [121] - 20 designs with LEF/DEF based global routing problem ICCAD'15 [122] - 8 superblue designs of incremental timing-driven placement problem ICCAD'04 [123] - IBM designs for mixed-size placement	Wirelength, Routing Congestion, DRC violation, IR drop & Timing
3	DAC [124]	An event for chips to systems. DAC'12 [125] - 10 superblue designs for routability-driven placement problem	Wirelength, Routing Congestion, DRC violation & Timing
4	MCNC [53]	Microelectronics Center of North Carolina (MCNC), a widely used benchmark sets in VLSI CAD Research	Wirelength
5	GSRC [54]	Gigascale Systems Research Center (GSRC), provides real-world larger designs compared to MCNC	Wirelength
6	CircuitNet [126]	An open-source public dataset intended for ML-based research in the VLSI CAD domain (28nm and 14nm tech node)	Routing Congestion, DRC violation, Timing & IR drop

[125], MCNC [53], and GSRC [54] benchmark designs have provided researchers with test designs for improving PnR optimization. These benchmark designs are processed using commercial EDA tools to generate samples that can be used to train and test the ML-DL model at different stages in the layout process [127]. Recently, an open-source dataset called CircuitNet [128] was introduced to facilitate learning-based research in Electronic Design Automation (EDA). It has more than 20,000 samples collected across various designs using Cadence Innovus, incorporating variations in logic synthesis and physical design parameters, and supports tasks such as routability, IR drop and timing prediction. The CircuitNet dataset can be accessed through a publicly available open repository [126].

However, certain constraints exist in the implementation of these designs in EDA tools. In particular, the compatibility issues with specific versions of CAD tools or technology nodes make it difficult to execute the complete PnR flow, which in turn limits the ability to accurately evaluate and optimize PnR algorithms.

VI. KEY CHALLENGES

Due to advanced technology nodes, PnR optimization becomes a more complex process that often requires significant time to meet all the design requirements. Thus, the need for ML-DL-assisted EDA tools has arisen in recent decades [129]. The key challenges in the deployment of learning-based PnR are addressed below.

A. DOMAIN-SPECIFIC KNOWLEDGE

During the PnR process, design metrics such as area, power, and delay should be properly balanced by adhering to the design rules. Traditionally, many heuristic algorithms have been employed for design optimization with the

expertise of the chip designers. ML-DL algorithms often act as data-driven opaque models and learn design correlations from training data without explicitly incorporating fundamental PnR principles. To develop effective learning-based algorithms, ML engineers have to understand the trade-offs in the PnR process and to model the algorithms accordingly. The integration of expert domain knowledge with DL-ML algorithms works well to achieve optimal placement solutions [5].

B. DATA IMBALANCE AND LIMITED DATA AVAILABILITY

The generation of training data is a major problem in learning-based PnR optimization. Deep Neural Networks (DNNs) often require a large amount of data for effective training. It is also crucial that the training data closely resemble real-world designs to ensure a reliable performance during deployment. Another issue is data imbalance, in which the majority of the data belongs to one or a few classes. To overcome the limited availability of data, training samples can be collected from various designs, and a data augmentation technique [74] can be performed. This is the formation of permutations from the available training data. Another way to increase data availability is through synthetic data generation, for instance, using a Generative Adversarial Network (GAN) to create realistic congestion feature maps [67]. These strategies help mitigate both data scarcity and data imbalance, thereby enhancing training efficiency and model accuracy.

C. TRANSFER LEARNING AND CROSS-NODE GENERALIZATION

A major challenge in learning-based PnR is to enable models trained on a particular technology node or benchmark dataset to effectively generalize to other nodes and unseen

designs. ML-DL algorithms trained on a specific dataset or process node often fail when deployed under different scenarios. Transfer learning has been explored as a promising solution, in which a model is first trained on one dataset and subsequently fine-tuned on a different dataset or related samples [130], [131]. For example, a model pretrained on a large dataset (CircuitNet) can then be fine-tuned on ISPD2015 designs or designs from different technology nodes [59]. However, cross-node generalization remains difficult, because each technology node differs in terms of standard cell libraries, routing resources, design rules, and constraints. These variations cause the models to overfit on specific technology nodes. To address this issue, advanced feature engineering, normalization strategies, and domain adaptation methods are required.

D. COMPUTATIONAL OVERHEAD

Training deep neural networks or graph-based models on large netlists requires extensive computational resources and long runtimes, particularly when handling millions of standard cells and complex connectivity structures. In addition to runtime, memory requirements are also a major constraint, as storing and processing large-scale circuit graphs and feature representations often exceeds the hardware capacity. These limitations restrict the applicability of state-of-the-art models to large-scale industrial design flows. Addressing this issue requires designing lightweight models that balance memory and computational efficiency while maintaining predictive accuracy.

VII. FUTURE DIRECTIONS

Despite recent advances in learning-based approaches for PnR, several challenges remain.

The following points outline potential directions that need to be addressed.

- Future research on learning-based PnR prediction and optimization should focus on building more robust and generalizable models that can adapt across different technology nodes and large-scale designs.
- Integrating domain-specific constraints into the learning process allows models to better capture the real implementation of designs.
- By leveraging graph-based and multi-scale representations, models can effectively capture both local and global spatial connectivity patterns in layout structures while adhering to critical design rules.
- Multi-objective optimization remains a key challenge, as PnR should simultaneously balance design PPA metrics. Approaches such as reinforcement learning, multitask learning, and hybrid optimization can enable models to learn trade-offs between these competing objectives.
- Another future aspect is the creation of large, realistic datasets derived from industrial flows across multiple

technology nodes, which are essential for advancing ML-DL model training and evaluation.

- Future work should address the high computational overhead and memory requirements of learning-based approaches to enable efficient scaling to large industrial designs while maintaining accuracy.
- The integration of learning-based methods and hardware accelerators across the PnR stages enables faster and more accurate design closures.

Thus, future research on learning-based PnR should concentrate on developing robust, generalizable models that can adapt to different technology nodes while simultaneously integrating domain-specific constraints and multi-objective optimization.

VIII. CONCLUSION

The utility of ML-DL algorithms is increasing nowadays in the back-end flow of ASIC design, as it minimizes the total runtime of the design as well as the workload of engineers. In this survey, a comprehensive overview of learning-based approaches for PnR optimization is provided, from the evolution of heuristic and analytical methods to recent advances in ML-DL models. This review examines the application of learning-based models to a wide spectrum of PnR objectives, performance factors, and the benchmarks necessary for reproducible and comparable research. Advances in graph-based learning, foundation models, and cross-design generalization can significantly enhance the accuracy and robustness of PnR predictions and optimization. Addressing key challenges such as scalability, data availability, and the integration of learning-based methods into existing EDA workflows is crucial for advancing research in this field.

REFERENCES

- [1] I. L. Markov, J. Hu, and M.-C. Kim, "Progress and challenges in VLSI placement research," *Proc. IEEE*, vol. 103, no. 11, pp. 1985–2003, Nov. 2015.
- [2] K. Shahookar and P. Mazumder, "VLSI cell placement techniques," *ACM Comput. Surveys*, vol. 23, no. 2, pp. 143–220, Jun. 1991.
- [3] Y. Qiu, Y. Xing, X. Zheng, P. Gao, S. Cai, and X. Xiong, "Progress of placement optimization for accelerating VLSI physical design," *Electronics*, vol. 12, no. 2, p. 337, Jan. 2023.
- [4] L. Li, Y. Cai, and Q. Zhou, "A survey on machine learning-based routing for VLSI physical design," *Integration*, vol. 86, pp. 51–56, Sep. 2022.
- [5] M. Rapp, H. Amrouch, Y. Lin, B. Yu, D. Z. Pan, M. Wolf, and J. Henkel, "MLCAD: A survey of research in machine learning for CAD keynote paper," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3162–3181, Oct. 2022.
- [6] N. Sharma, R. Sharma, and N. Jindal, "Machine learning and deep learning applications—A vision," in *Global Transitions Proc.*, 2021, vol. 2, no. 1, pp. 24–28.
- [7] A. Younesi, M. Ansari, M. Fazli, A. Ejlali, M. Shafique, and J. Henkel, "A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends," *IEEE Access*, vol. 12, pp. 41180–41218, 2024.
- [8] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. 18th Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, Munich, Germany, 2015, pp. 234–241.
- [9] V. P. Dwivedi, C. K. Joshi, A. T. Luu, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking graph neural networks," *J. Mach. Learn. Res.*, vol. 24, no. 43, pp. 1–48, 2023.

- [10] U. Kamath, K. Graham, and W. Emara, *Transformers for Machine Learning: A Deep Dive*. Boca Raton, FL, USA: CRC Press, 2022.
- [11] K. Bayoudh, "A survey of multimodal hybrid deep learning for computer vision: Architectures, applications, trends, and challenges," *Inf. Fusion*, vol. 105, May 2024, Art. no. 102217.
- [12] A. Bunglowala and B. Singhi, "Performance evaluation and comparison and improvement of standard cell placement techniques in VLSI design," in *Proc. 1st Int. Conf. Emerg. Trends Eng. Technol.*, 2008, pp. 468–473.
- [13] M. Mir, "Application of simulated annealing to cluster-boundary search algorithm for macrocell placement optimization," in *Proc. Int. Conf. Comput. Commun. Eng. (ICCCCE)*, May 2010, pp. 1–4.
- [14] P. Subbaraj, S. S. Sankar, and S. Anand, "Parallel genetic algorithm for VLSI standard cell placement," in *Proc. Int. Conf. Adv. Comput., Control, Telecommun. Technol.*, Dec. 2009, pp. 80–84.
- [15] S.-T. Hsieh, C.-W. Lin, and T.-Y. Sun, "Particle swarm optimization for macrocell overlap removal and placement," in *Proc. IEEE Swarm Intell. Symp., SIS.*, Jun. 2005, pp. 177–180.
- [16] G. A. Tagliarini, J. F. Christ, and E. W. Page, "Optimization using neural networks," *IEEE Trans. Comput.*, vol. 40, no. 12, pp. 1347–1358, Dec. 1991.
- [17] M.-C. Kim, D.-J. Lee, and I. L. Markov, "SimPL: An effective placement algorithm," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2010, pp. 649–656.
- [18] M.-C. Kim, N. Viswanathan, C. J. Alpert, I. L. Markov, and S. Ramji, "MAPLE: Multilevel adaptive placement for mixed-size designs," in *Proc. ACM Int. Symp. Int. Symp. Phys. Design*, New York, NY, USA, Mar. 2012, pp. 193–200.
- [19] T. Lin, C. Chu, J. R. Shinnerl, I. Bustany, and I. Nedelchev, "POLAR: Placement based on novel rough legalization and refinement," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2013, pp. 357–362.
- [20] C.-K. Cheng, A. B. Kahng, I. Kang, and L. Wang, "RePLAce: Advancing solution quality and routability validation in global placement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 9, pp. 1717–1730, Sep. 2019.
- [21] J. Lu, H. Zhuang, P. Chen, H. Chang, C.-C. Chang, Y.-C. Wong, L. Sha, D. Huang, Y. Luo, C.-C. Teng, and C.-K. Cheng, "EPlace-MS: Electrostatics-based placement for mixed-size circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 5, pp. 685–698, May 2015.
- [22] P. Spindler, U. Schlichtmann, and F. M. Johannes, "Kraftwerk2—A fast force-directed quadratic placement approach using an accurate net model," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 8, pp. 1398–1411, Aug. 2008.
- [23] N. Viswanathan, M. Pan, and C. Chu, "FastPlace 3.0: A fast multilevel quadratic placement algorithm with placement congestion control," in *Proc. Asia South Pacific Design Autom. Conf.*, Jan. 2007, pp. 135–140.
- [24] N. Viswanathan, G.-J. Nam, C. J. Alpert, P. Villarrubia, H. Ren, and C. Chu, "RQL: Global placement via relaxed quadratic spreading and linearization," in *Proc. 44th ACM/IEEE Design Autom. Conf.*, Jun. 2007, pp. 453–458.
- [25] A. B. Kahng and Q. Wang, "A faster implementation of APlace," in *Proc. Int. Symp. Phys. design*, New York, NY, USA, Apr. 2006, pp. 218–220.
- [26] T. F. Chan, J. Cong, J. R. Shinnerl, K. Sze, and M. Xie, "mpl6: Enhanced multilevel mixed-size placement," in *Proc. Int. Symp. Phys. Design*, New York, NY, USA, 2006, pp. 212–214.
- [27] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, "NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1228–1240, Jul. 2008.
- [28] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI Physical Design: From Graph Partitioning to Timing Closure*, 1st ed., Cham, Switzerland: Springer, 2011.
- [29] J. Hu and S. S. Sapatnekar, "A survey on multi-net global routing for integrated circuits," *Integration*, vol. 31, no. 1, pp. 1–49, Nov. 2001.
- [30] C. Y. Lee, "An algorithm for path connections and its applications," *IRE Trans. Electron. Comput.*, vols. EC–10, no. 3, pp. 346–365, Sep. 1961.
- [31] M. Pan and C. Chu, "FastRoute 2.0: A high-quality and efficient global router," in *Proc. Asia South Pacific Design Autom. Conf.*, Jan. 2007, pp. 250–255.
- [32] R. Kastner, E. Bozorgzadeh, and M. Sarrafzadeh, "Pattern routing: Use and theory for increasing predictability and avoiding coupling," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 21, no. 7, pp. 777–790, Jul. 2002.
- [33] C. Chu and Y.-C. Wong, "FLUTE: Fast lookup table based rectilinear Steiner minimal tree algorithm for VLSI design," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 1, pp. 70–83, Jan. 2008.
- [34] Q. Zhu, H. Zhou, T. Jing, X.-L. Hong, and Y. Yang, "Spanning graph-based nonrectilinear Steiner tree algorithms," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 24, no. 7, pp. 1066–1075, Jul. 2005.
- [35] W.-K. Chow, L. Li, E. F. Y. Young, and C.-W. Sham, "Obstacle-avoiding rectilinear Steiner tree construction in sequential and parallel approach," *Integration*, vol. 47, no. 1, pp. 105–114, Jan. 2014.
- [36] A. Youssef, Z. Yang, M. Anis, S. Areibi, A. Vannelli, and M. Elmasry, "A power-efficient multipin ILP-based routing technique," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 1, pp. 225–235, Jan. 2010.
- [37] C.-J. Chang, P.-J. Huang, T.-C. Chen, and C. J. Liu, "ILP-based inter-die routing for 3D ICs," in *Proc. 16th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2011, pp. 330–335.
- [38] R. T. Hadsell and P. H. Madden, "Improved global routing through congestion estimation," in *Proc. Design Autom. Conf.*, Jun. 2003, pp. 28–31.
- [39] H. Tang, G. Liu, X. Chen, and N. Xiong, "A survey on Steiner tree construction and global routing for VLSI design," *IEEE Access*, vol. 8, pp. 68593–68622, 2020.
- [40] Y. Zhang, Y. Xu, and C. Chu, "FastRoute3.0: A fast and high quality global router based on virtual capacity," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design*, Nov. 2008, pp. 344–349.
- [41] M. D. Moffitt, "MaizeRouter: Engineering an effective global router," in *Proc. Asia South Pacific Design Autom. Conf.*, Jan. 2008, pp. 226–231.
- [42] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao, "NCTU-GR 2.0: Multithreaded collision-aware global routing with bounded-length maze routing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 5, pp. 709–722, May 2013.
- [43] M. Cho, K. Lu, K. Yuan, and D. Z. Pan, "BoxRouter 2.0: A hybrid and robust global router with layer assignment for routability," *ACM Trans. Design Autom. Electron. Syst.*, vol. 14, no. 2, pp. 1–21, Apr. 2009.
- [44] Y. Xu, Y. Zhang, and C. Chu, "FastRoute 4.0: Global router with efficient via minimization," in *Proc. Asia South Pacific Design Autom. Conf.*, Jan. 2009, pp. 576–581.
- [45] Tapan Gupta, *Copper Interconnect Technology*. Cham, Switzerland: Springer, 2010.
- [46] A. B. Amin and M. S. Ullah, "Mathematical framework of tetramorphic MWCNT configuration for VLSI interconnect," *IEEE Trans. Nanotechnol.*, vol. 19, pp. 749–759, 2020.
- [47] A. B. Amin, S. M. Shakil, and M. S. Ullah, "A theoretical modeling of adaptive mixed CNT bundles for high-speed VLSI interconnect design," *Crystals*, vol. 12, no. 2, p. 186, Jan. 2022.
- [48] W.-K. Cheng, Y.-Y. Guo, and C.-S. Wu, "Evaluation of routability-driven macro placement with machine-learning technique," in *Proc. 7th Int. Symp. Next Gener. Electron. (ISNE)*, May 2018, pp. 1–3.
- [49] N. Viswanathan, S.-H. Huang, R.-B. Lin, and M.-C. Kim, "Overview of the 2015 CAD contest at ICCAD," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2015, pp. 910–911.
- [50] G.-J. Nam, "ISPD 2006 placement contest: Benchmark suite and results," in *Proc. Int. Symp. Phys. design*, Apr. 2006, p. 167.
- [51] Y. Liu, Z. Ju, Z. Li, M. Dong, H. Zhou, J. Wang, F. Yang, X. Zeng, and L. Shang, "GraphPlanner: Floorplanning with graph neural network," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 2, pp. 1–24, Mar. 2023.
- [52] Q. Xu, H. Geng, S. Chen, B. Yuan, C. Zhuo, Y. Kang, and X. Wen, "GoodFloorplan: Graph convolutional network and reinforcement learning-based floorplanning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3492–3502, Oct. 2022.
- [53] *MCNC Benchmark*. Accessed: May 26, 2007. [Online]. Available: <http://vlsicad.eecs.umich.edu/BK/MCNCbench/>
- [54] *GSRC Benchmark*. Accessed: May 26, 2007. [Online]. Available: <http://vlsicad.eecs.umich.edu/BK/GSRCbench/>
- [55] Y.-C. Lu, S. Pentapati, and S. K. Lim, "The law of attraction: Affinity-aware placement optimization using graph neural networks," in *Proc. Int. Symp. Phys. Design*, Mar. 2021, pp. 7–14.

- [56] Y.-C. Lu, T. Yang, S. K. Lim, and H. Ren, "Placement optimization via PPA-directed graph clustering," in *Proc. ACM/IEEE 4th Workshop Mach. Learn. CAD (MLCAD)*, Sep. 2022, pp. 1–6.
- [57] A. Agnesina, K. Chang, and S. K. Lim, "Parameter optimization of VLSI placement through deep reinforcement learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 4, pp. 1295–1308, Apr. 2023.
- [58] M. Amini, Z. Zhang, S. Penmetsa, Y. Zhang, J. Hao, and W. Liu, "Generalizable floorplanner through corner block list representation and hypergraph embedding," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 2692–2702.
- [59] Q. Cai, S. Zhang, P. Ding, X. Wu, P. Huang, T. Zhang, H. You, Z. Cheng, and Q. Cui, "ST-FPN: A Swin transformer-based lightweight model for accurate VLSI congestion prediction," *J. Supercomput.*, vol. 81, no. 11, pp. 1–23, Jul. 2025.
- [60] H. Li, Y. Huo, Y. Wang, X. Yang, M. Hao, and X. Wang, "A lightweight inception boosted U-Net neural network for routability prediction," in *Proc. 2nd Int. Symp. Electron. Design Autom. (ISED)*, May 2024, pp. 648–653.
- [61] Y. Chen, D. Zhang, and H. Chen, "A standard deviation incorporated attention module enhanced U-Net for routability prediction," in *Proc. 9th Int. Conf. Electron. Technol. Inf. Sci. (ICETIS)*, May 2024, pp. 671–674.
- [62] R. Liu, G. Liu, and X. Lei, "MSANet: Congestion prediction for chip routing based on multi-scale attention network," in *Proc. 3rd Int. Symp. Semiconductor Electron. Technol. (ISSET)*, Aug. 2024, pp. 587–592.
- [63] B. Wang, G. Shen, D. Li, J. Hao, W. Liu, Y. Huang, H. Wu, Y. Lin, G. Chen, and P. A. Heng, "LHNN: Lattice hypergraph neural network for VLSI congestion prediction," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 1297–1302.
- [64] R. Kirby, S. Godil, R. Roy, and B. Catanzaro, "CongestionNet: Routing congestion prediction using deep graph neural networks," in *Proc. IFIP/IEEE 27th Int. Conf. Very Large Scale Integr. (VLSI-SoC)*, Oct. 2019, pp. 217–222.
- [65] K. Min, S. Kwon, S.-Y. Lee, D. Kim, S. Park, and S. Kang, "ClusterNet: Routing congestion prediction and optimization using netlist clustering and graph neural networks," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [66] X. Peng, Y. Zou, X. Song, Y. Liu, X. Chen, S. Liu, and X. Cheng, "Leveraging Steiner tree grid segment in GNN for VLSI pre-routing congestion prediction," in *Proc. Int. Symp. Electron. Design Autom. (ISED)*, May 2025, pp. 387–392.
- [67] G. Mo, Y. Xia, J. Ou, S. Cai, and X. Xiong, "Layout congestion prediction based on regression-ViT," *ACM Trans. Design Autom. Electron. Syst.*, vol. 30, no. 1, pp. 1–21, Jan. 2025.
- [68] S. Zheng, L. Zou, P. Xu, S. Liu, B. Yu, and M. Wong, "Lay-Net: Grafting netlist knowledge on layout-based congestion prediction," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
- [69] H. Gu, Y. Wang, X. Zheng, K. Peng, Z. Zhu, J. Chen, and J. Yang, "Dual multimodal fusions with convolution and transformer layers for VLSI congestion prediction," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 6, pp. 2378–2391, Jun. 2025.
- [70] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany, and D. Z. Pan, "DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, pp. 1–6.
- [71] Y.-Y. Huang, C.-T. Lin, W.-L. Liang, and H.-M. Chen, "Learning based placement refinement to reduce DRC short violations," in *Proc. Int. Symp. VLSI Design, Autom. Test (VLSI-DAT)*, Apr. 2021, pp. 1–4.
- [72] Z. Xie, Y.-H. Huang, G.-Q. Fang, H. Ren, S.-Y. Fang, Y. Chen, and J. Hu, "RouteNet: Routability prediction for mixed-size designs using convolutional neural network," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, Nov. 2018, pp. 1–8.
- [73] Y.-H. Huang, Z. Xie, G.-Q. Fang, T.-C. Yu, H. Ren, S.-Y. Fang, Y. Chen, and J. Hu, "Routability-driven macro placement with embedded CNN-based prediction model," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 180–185.
- [74] R. Liang, H. Xiang, D. Pandey, L. N. Reddy, S. Ramji, G.-J. Nam, and J. Hu, "Design rule violation prediction at sub-10-nm process nodes using customized convolutional networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3503–3514, Oct. 2022.
- [75] X. Chen, Z. Di, W. Wu, Q. Wu, J. Shi, and Q. Feng, "Detailed routing short violation prediction using graph-based deep learning model," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 2, pp. 564–568, Feb. 2022.
- [76] K. Baek, H. Park, S. Kim, K. Choi, and T. Kim, "Pin accessibility and routing congestion aware DRC hotspot prediction using graph neural network and U-Net," in *Proc. 41st IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2022, pp. 1–9.
- [77] J. Lin, W. Lin, S. Liang, P. Gao, Y. Xing, T. Wu, X. Xiong, and S. Cai, "An efficient method of DRC violation prediction with a serial deep learning model," *ACM Trans. Design Autom. Electron. Syst.*, vol. 29, no. 6, pp. 1–16, Nov. 2024.
- [78] J. Lin, S. Liang, W. Lin, P. Gao, Y. Xing, T. Wu, X. Xiong, and S. Cai, "Early stage DRC hotspot prediction for mixed-size designs through an efficient graph-based deep learning," *ACM Trans. Design Autom. Electron. Syst.*, vol. 30, no. 4, pp. 1–21, Jul. 2025.
- [79] L. Bai and L. Chen, "Machine-learning-based early-stage timing prediction in SoC physical design," in *Proc. 14th IEEE Int. Conf. Solid-State Integr. Circuit Technol. (ICSIT)*, Oct. 2018, pp. 1–3.
- [80] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin, "A timing engine inspired graph neural network model for pre-routing slack prediction," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 1207–1212.
- [81] P. Cao, Z. Li, and W. Ding, "A GNN-based placement optimization guidance framework by physical and timing prediction," *Electronics*, vol. 14, no. 2, p. 329, Jan. 2025.
- [82] Z. Wang, S. Liu, Y. Pu, S. Chen, T.-Y. Ho, and B. Yu, "PRO-TIME: Pre-routing optimization-aware timing prediction via multimodal learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11002564>
- [83] M. He, L. Tian, Y. Jia, Y. Ye, and L. Shi, "Pre-routing timing estimation considering power delivery network," in *Proc. Int. Symp. Electron. Design Autom. (ISED)*, May 2025, pp. 281–286.
- [84] D. Lim and H. Park, "Timing-driven detailed placement with unsupervised graph learning," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.
- [85] *OpenDP Open Source Detailed Placement Engine*. Accessed: May 15, 2025. [Online]. Available: <https://github.com/sanggido/OpenDP>
- [86] H. Liu, Z. Zeng, S. Tao, Z. Huang, Y. Li, B. Xie, W. Gao, and X. Li, "AiTPO: KAN-UNet heterogeneous network for timing prediction and optimization at global routing," *ACM Trans. Design Autom. Electron. Syst.*, vol. 2025, pp. 1–28, May 2025.
- [87] Z. Lin, H. Zhang, P. Gao, F. Yu, T. Wu, X. Xiong, and S. Cai, "GNN-based timing prediction in prerouting stage with multitask learning strategy," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 8, pp. 3154–3164, Aug. 2025.
- [88] S. Bodhe, Z. Zhang, A. Hamidzadeh, S. Kai, Y. Zhang, and M. Yuan, "E2ESlack: An end-to-end graph-based framework for pre-routing slack prediction," 2025, *arXiv:2501.07564*.
- [89] Y. Xing, H. Hu, W. Li, S. Cai, and X. Xiong, "Concurrent prediction of timing and wire length using a multi-task graph neural network," *ACM Trans. Design Autom. Electron. Syst.*, vol. 30, no. 4, pp. 1–20, Jul. 2025.
- [90] V. A. Chhabria, A. B. Kahng, M. Kim, U. Mallappa, S. S. Sapatnekar, and B. Xu, "Template-based PDN synthesis in floorplan and placement using classifier and CNN techniques," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 44–49.
- [91] Z. Xie, H. Ren, B. Khailany, Y. Sheng, S. Santosh, J. Hu, and Y. Chen, "PowerNet: Transferable dynamic IR drop estimation via maximum convolutional neural network," in *Proc. 25th Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2020, pp. 13–18.
- [92] V. A. Chhabria, Y. Zhang, H. Ren, B. Keller, B. Khailany, and S. S. Sapatnekar, "MAVIREC: ML-aided vectored IR-drop estimation and classification," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1825–1828.
- [93] P.-W. Huang and Y.-W. Chang, "Routability-driven power/ground network optimization based on machine learning," *ACM Trans. Design Autom. Electron. Syst.*, vol. 28, no. 4, pp. 1–27, Jul. 2023.
- [94] Y.-T. Liu, Y.-H. Cheng, S.-Y. Wu, and H.-M. Chen, "CFIRSTNET: Comprehensive features for static IR drop estimation with neural network," in *Proc. 43rd IEEE/ACM Int. Conf. Comput.-Aided Design*, Oct. 2024, pp. 1–9.
- [95] P. Li, Y. Zuo, Y. Sun, H. Yan, and L. Shi, "ICDaIR: Distribution-aware static IR drop prediction flow based on image classification," in *Proc. ACM/IEEE 6th Symp. Mach. Learn. CAD (MLCAD)*, Sep. 2024, pp. 1–6.

- [96] A. Mondal and T. Samanta, "Artificial neural network based prediction model for IR drop measurement in a VLSI power delivery network," *J. Electron. Test.*, vol. 41, no. 3, pp. 387–406, Jun. 2025.
- [97] Y. Zuo, P. Li, Y. Sun, H. Yan, and L. Shi, "Enhanced TransUNet framework for predicting static IR drop and chip routability," *ACM Trans. Design Autom. Electron. Syst.*, Jul. 2025, doi: 10.1145/3750726.
- [98] C. Yu, Y. Teng, W. Dai, Y. Li, W. W. Xing, X. Wu, D. Niu, and Z. Jin, "LaRED: Efficient IR drop predictor with layout-preserving rebuilder-encoder-decoder architecture," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.
- [99] C.-C. Lan, C.-C. Su, Y.-H. Lu, and Y.-W. Chang, "Robust technology-transferable static IR drop prediction based on image-to-image machine learning," in *Proc. 30th Asia South Pacific Design Autom. Conf.*, Jan. 2025, pp. 1399–1405.
- [100] T. Mohaidat and K. Khalil, "A survey on neural network hardware accelerators," *IEEE Trans. Artif. Intell.*, vol. 5, no. 8, pp. 3801–3822, Aug. 2024.
- [101] R. Hao, Y. Cai, and Q. Zhou, "Intelligent and kernelized placement: A survey," *Integration*, vol. 86, pp. 44–50, Sep. 2022.
- [102] G.-J. Nam, C. J. Alpert, P. Villarrubia, B. Winter, and M. Yildiz, "The ISPD2005 placement contest and benchmark suite," in *Proc. Int. Symp. Phys. design*, New York, NY, USA, Apr. 2005, pp. 216–220.
- [103] S. Liu, Q. Sun, P. Liao, Y. Lin, and B. Yu, "Global placement with deep learning-enabled explicit routability optimization," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Feb. 2021, pp. 1821–1824.
- [104] Y.-C. Lu, H. Ren, H.-H. Hsiao, and S. K. Lim, "DREAM-GAN: Advancing DREAMPlace towards commercial-quality using generative adversarial learning," in *Proc. Int. Symp. Phys. Design*, Mar. 2023, pp. 141–148.
- [105] Z. Guo and Y. Lin, "Differentiable-timing-driven global placement," in *Proc. 59th ACM/IEEE Design Autom. Conf.*, Jul. 2022, pp. 1315–1320.
- [106] P. Liao, D. Guo, Z. Guo, S. Liu, Y. Lin, and B. Yu, "DREAMPlace 4.0: Timing-driven placement with momentum-based net weighting and Lagrangian-based refinement," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 10, pp. 3374–3387, Oct. 2023.
- [107] Y. Han, K. Chakraborty, and S. Roy, "A global router on GPU architecture," in *Proc. IEEE 31st Int. Conf. Comput. Design (ICCD)*, Oct. 2013, pp. 78–84.
- [108] S. Lin, J. Liu, E. F. Y. Young, and M. D. F. Wong, "GAMER: GPU-accelerated maze routing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 583–593, Feb. 2023.
- [109] J. Liu, C.-W. Pui, F. Wang, and E. F. Y. Young, "CUGR: Detailed-routability-driven 3D global routing with probabilistic resource model," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
- [110] Q. Wang, Y. Ma, K. Zhao, and Y. Tian, "A comprehensive survey of loss functions in machine learning," *Ann. Data Sci.*, vol. 9, no. 2, pp. 187–212, Apr. 2022.
- [111] D. Choi, C. J. Shallue, Z. Nado, J. Lee, C. J. Maddison, and G. E. Dahl, "On empirical comparisons of optimizers for deep learning," 2019, *arXiv:1910.05446*.
- [112] O. Rainio, J. Teuhio, and R. Klén, "Evaluation metrics and statistical tests for machine learning," *Sci. Rep.*, vol. 14, no. 1, p. 6086, Mar. 2024.
- [113] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," 2020, *arXiv:2010.16061*.
- [114] *ISPD International Symposium for Physical Design*. Accessed: May 21, 2025. [Online]. Available: <https://ispd.cc/ispd2026/index.php?page=contests>
- [115] *ISPD'19 Detailed Routing*. Accessed: May 21, 2025. [Online]. Available: <https://www.ispd.cc/contests/19/>
- [116] *ISPD'15 Blockage-Aware Detailed Routing-Driven Placement*. Accessed: May 21, 2025. [Online]. Available: https://www.ispd.cc/contests/15/ispd2015_contest.html
- [117] *ISPD'11 Routability-Driven Placement*. Accessed: May 21, 2025. [Online]. Available: https://www.ispd.cc/contests/11/ispd2011_contest.html
- [118] *ISPD'05 Placement*. Accessed: May 21, 2025. [Online]. Available: <https://www.ispd.cc/contests/05/contest.htm>
- [119] *ICCAD International Conference on Computer-Aided Design*. Accessed: May 21, 2025. [Online]. Available: <https://www.iccad-contest.org/History.html>
- [120] *ICCAD23 Began Benchmarks*. Accessed: May 21, 2025. [Online]. Available: <https://github.com/UMN-EDA/BeGAN-benchmarks>
- [121] *ICCAD19 Open-Source Global Route*. Accessed: May 21, 2025. [Online]. Available: <https://www.iccad-contest.org/2019/problems.html>
- [122] *ICCAD15 Incremental Timing-Driven Placement*. Accessed: May 21, 2025. [Online]. Available: <https://www.iccad-contest.org/2015/problemzC/default.html>
- [123] *ICCAD04 Mixed Size Placement Contest*. Accessed: May 21, 2025. [Online]. Available: <http://vlsicad.eecs.umich.edu/BK/ICCAD04bench/>
- [124] *DAC Design Automation Conference*. Accessed: May 21, 2025. [Online]. Available: <https://www.dac.com/>
- [125] *DAC2012 Routability-Driven Placement*. Accessed: May 21, 2025. [Online]. Available: http://archive.sigda.org/dac2012/contest/dac2012_contest/
- [126] (2022). *CircuitNet Open-Source Dataset*. [Online]. Available: <https://circuitnet.github.io/>
- [127] D. Amuru, A. Zahra, H. V. Vudumula, P. K. Cherupally, S. R. Gurram, A. Ahmad, and Z. Abbas, "AI/ML algorithms and applications in VLSI design and technology," *Integration*, vol. 93, Nov. 2023, Art. no. 102048.
- [128] Z. Chai, Y. Zhao, Y. Lin, W. Liu, R. Wang, and R. Huang, "CircuitNet: An open-source dataset for machine learning applications in electronic design automation (EDA)," 2022, *arXiv:2208.01040*.
- [129] J. Yan, X. Lyu, R. Cheng, and Y. Lin, "Towards machine learning for placement and routing in chip design: A methodological overview," 2022, *arXiv:2202.13564*.
- [130] R. Netto, S. Fabre, T. A. Fontana, V. Livramento, L. L. Pilla, L. Behjat, and J. L. Güntzel, "Algorithm selection framework for legalization using deep convolutional neural networks and transfer learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 5, pp. 1481–1494, May 2022.
- [131] S. Niu, Y. Liu, J. Wang, and H. Song, "A decade survey of transfer learning (2010–2020)," *IEEE Trans. Artif. Intell.*, vol. 1, no. 2, pp. 151–166, Oct. 2020.



R. JENILA received the B.E. degree in ECE from the St. Xavier's Catholic College of Engineering, Tamil Nadu, India, in 2017, and the M.E. degree in applied electronics from the Bethlahem Institute of Engineering, Tamil Nadu, in 2019. She is currently pursuing the Ph.D. degree with the School of Electronics Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu. Her research interests include RTL design, machine learning in physical design, and automation.



K. JAGANNADHA NAIDU (Senior Member, IEEE) received the B.Tech. degree in electronics and communication engineering from JNTU, India, and the M.Tech. degree in VLSI design and the Ph.D. degree from Vellore Institute of Technology, Vellore. Currently, he is an Associate Professor with Vellore Institute of Technology. He published 14 journal articles and six conference papers. His research interests include low-power high-performance VLSI architectures design and ML in EDA.

...